

数字人实践体验

主讲人：高林

时间：2026年6月17日

AI 生成图片



AI 重建 3D 场景





上一节课，我们拆解了数字人

1 数字世界的基础

中西计算机的发展



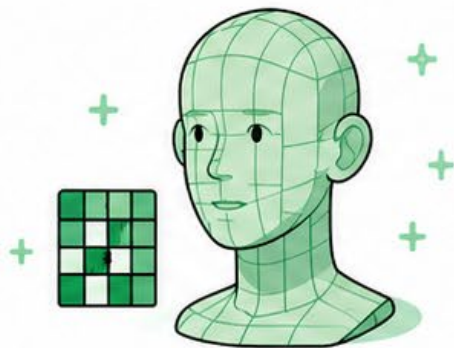
2 电脑如何画出世界

计算机图形学发展



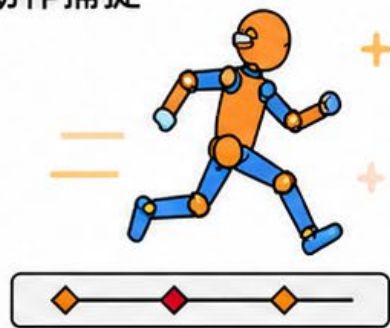
3 数字人的外形

三角形、模型、纹理、材质



4 数字人的动作

骨架、关键帧、插值、动作捕捉



这些方法大多需要**人工建模、人工调参、人工制作。**



那么，**AI** 能不能帮我们**自动生成**数字人？



课程提纲

这节课，我们一起看看 AI 如何帮助我们更快制作数字人。

1 数字人回顾

基础 / 外形 / 动作



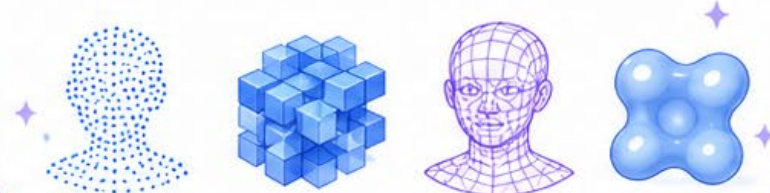
2 三维重建与3D扫描

扫描方式 / 流程 / 应用



3 几何表征方法

点云 / 体素 / 网格 / 隐式场



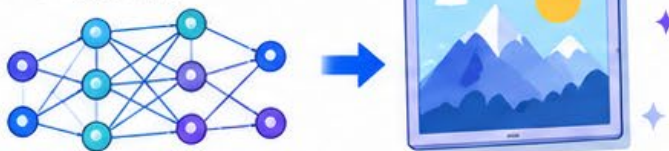
4 AI如何更快创建3D

相机数据 / 图像重建 / 真实场景



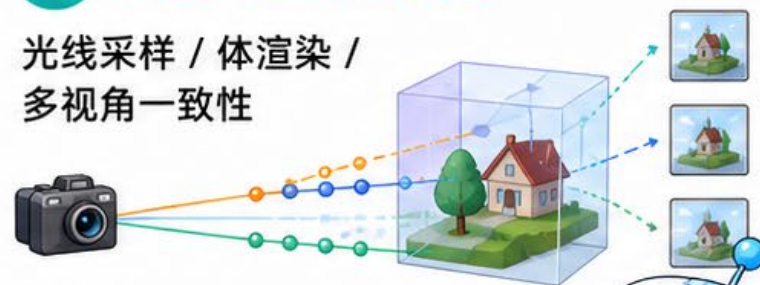
5 神经网络

神经网络基础 / AI生成图片



6 NeRF重建原理

光线采样 / 体渲染 / 多视角一致性



从3D扫描到NeRF，我们一步步走进数字人的AI世界。





三维重建：3D扫描

通过传感器捕捉真实物体的表面信息
获取空间坐标、颜色、纹理等数据

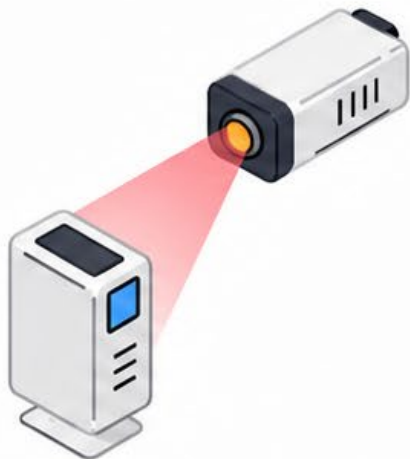
结构光扫描

投射已知图案，计算变形获取深度



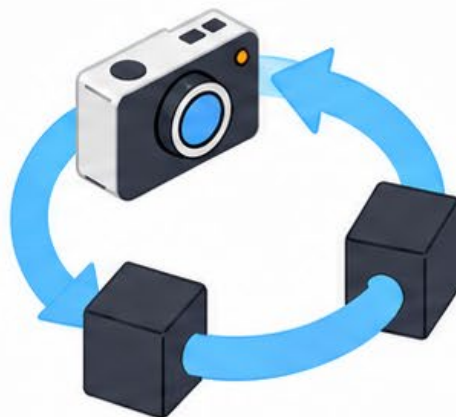
激光扫描

发射激光测距，逐点扫描



拍照重建

多角度拍照，图像匹配还原3D



RGB-D相机

同时获取颜色和深度信息



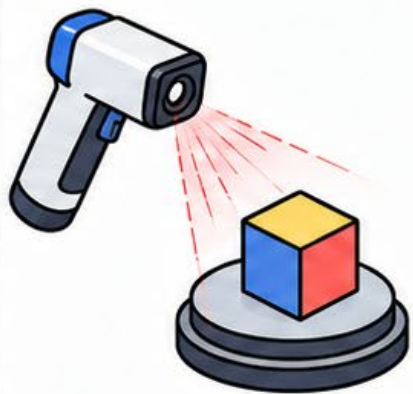


3D扫描的一般流程

把真实物体一步步变成数字模型：
先采集数据，再重建形状，最后生成可用的3D模型

1 数据采集

相机/传感器记录表面信息



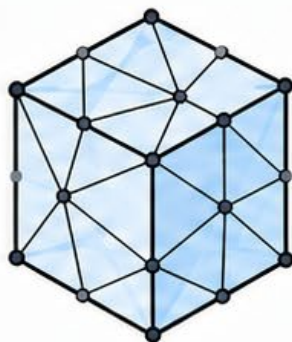
2 点云生成

得到很多空间采样点



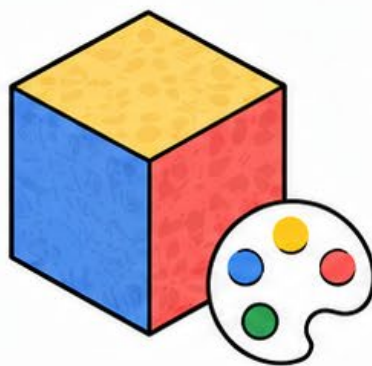
3 网格重建

把点连接成三角面



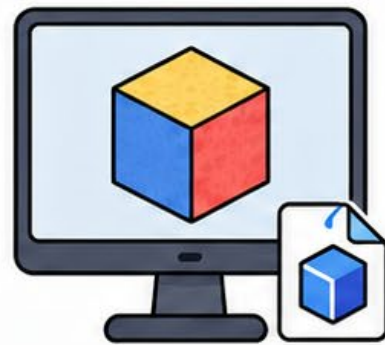
4 纹理贴图

把颜色和纹理贴回模型



5 模型输出

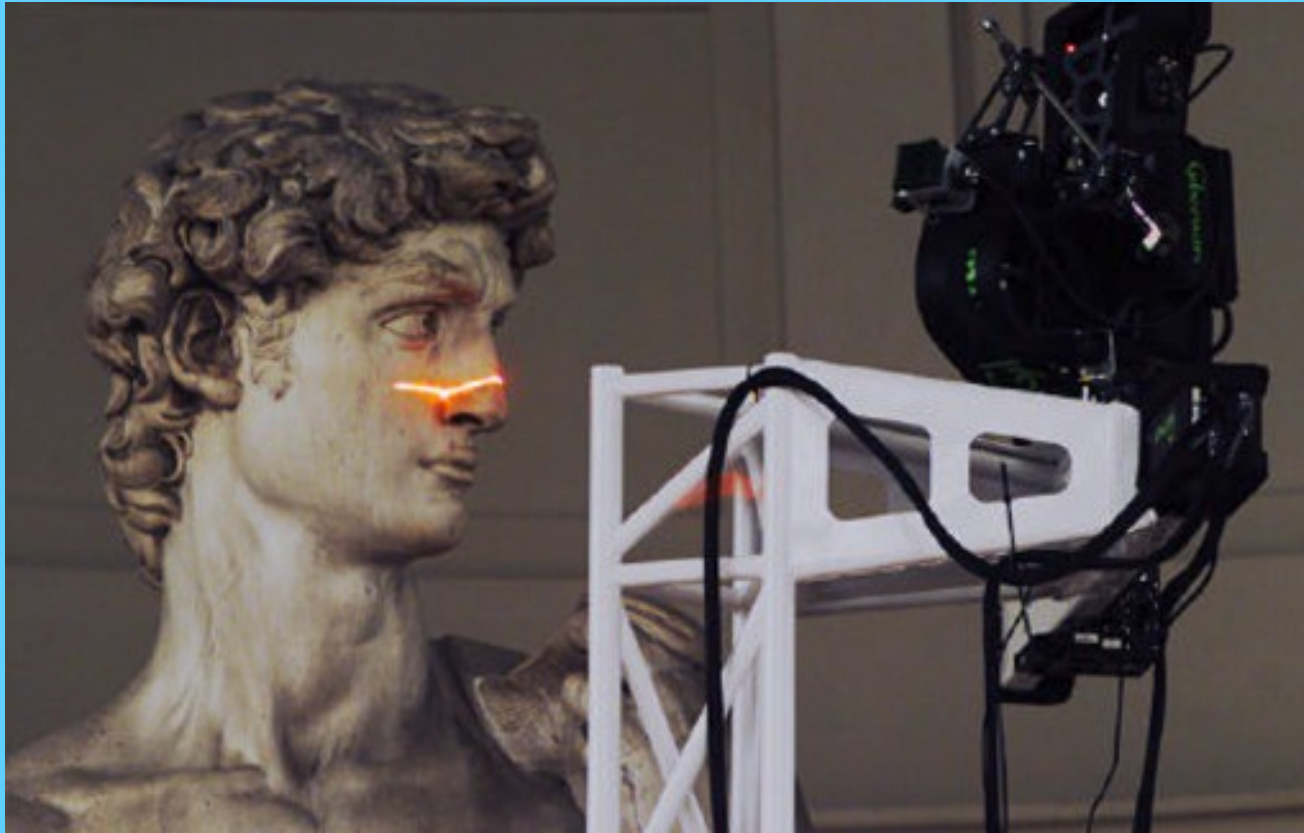
得到可编辑、可展示的
3D模型



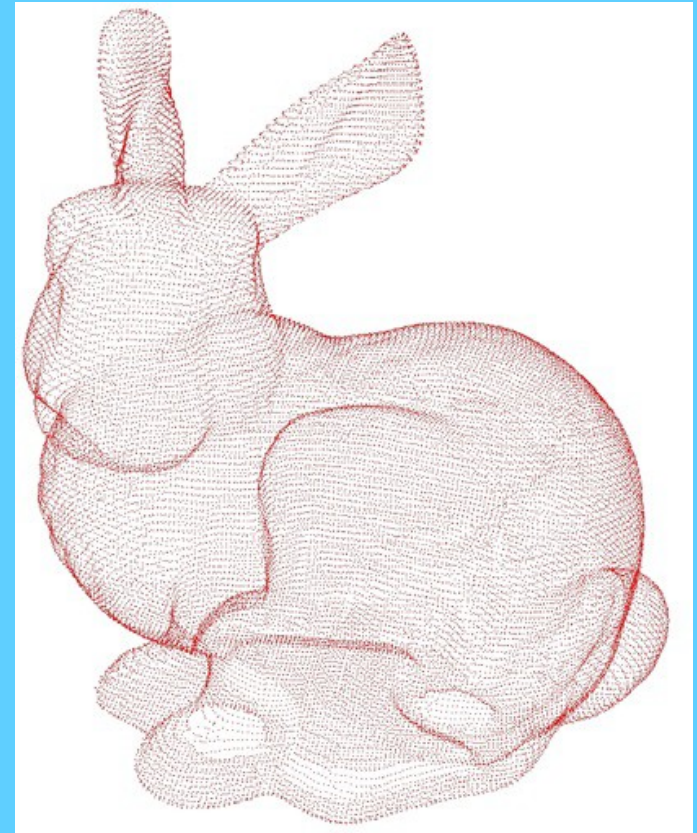
核心理解：3D扫描 = 采集真实世界的信息 + 在电脑中重建形状和外观

几何表征方法 - 点云

- 点云是最简单的表示: 只有点, 没有连接性。
- (x, y, z) 坐标的集合, 可能有法向信息。



激光扫描仪



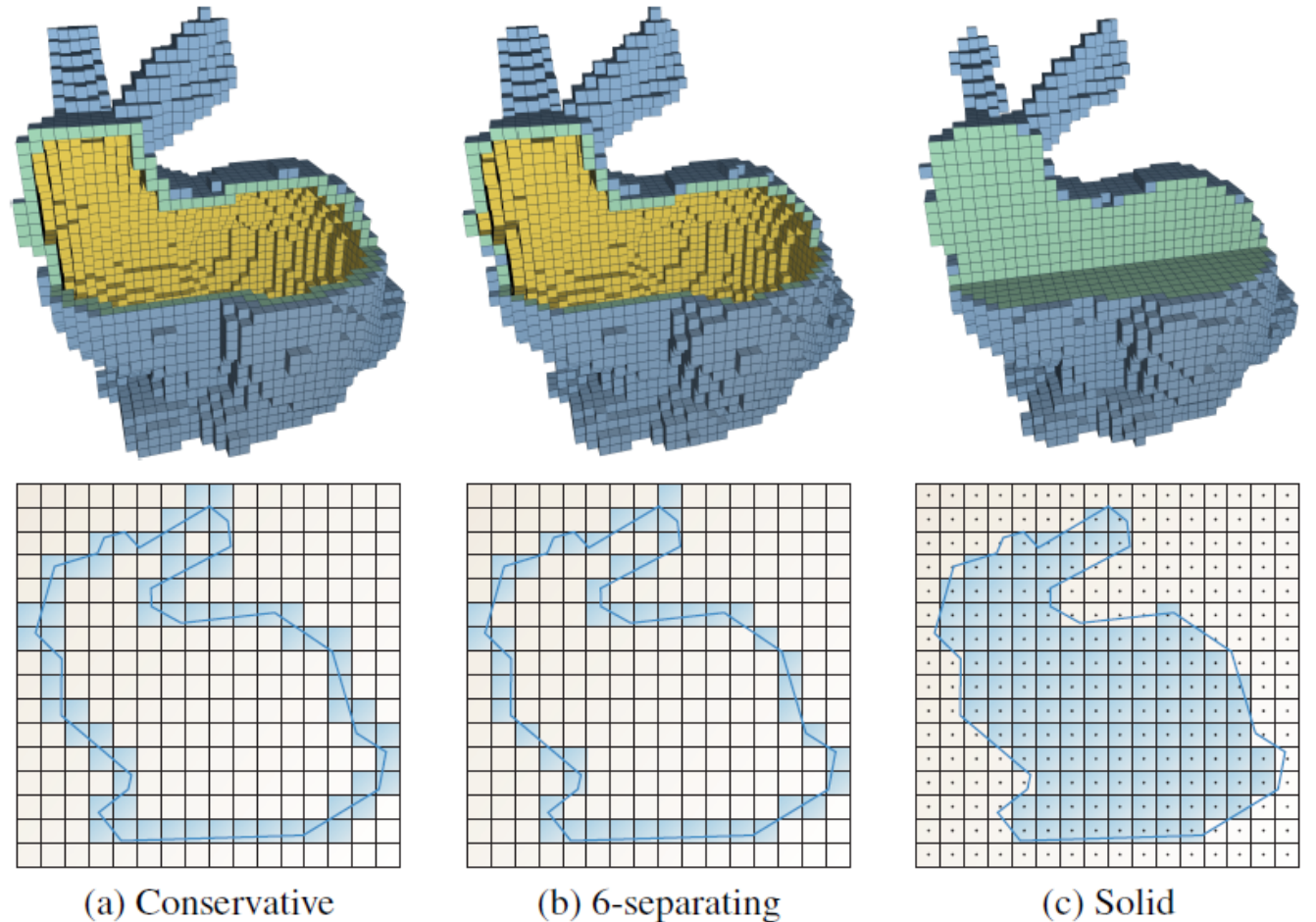
点云

几何表征方法 – 体素

- 体素 (voxel) 是体积像素 (volume pixel) 的简称,
- 类似于二维空间中的最小单位像素, 体素是三维空间分割中的最小单位



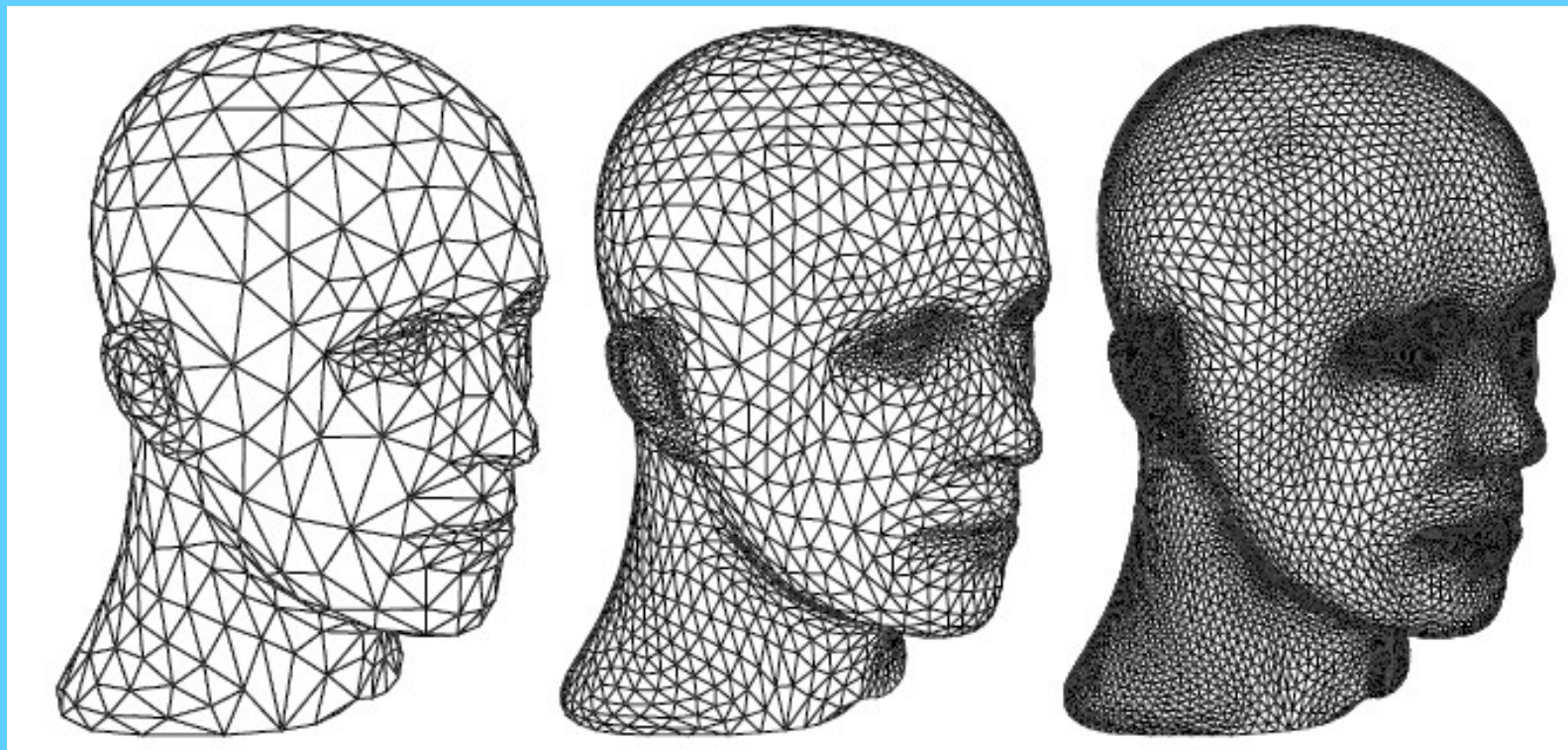
$$flag = \begin{cases} 0, & \text{apart} \\ 1, & \text{intersection} \end{cases}$$



几何表征方法 - 网格

- 网格的描述

- 一系列的面片 $F = (f_1, f_2, \dots, f_n)$, 每一个面片都是三角形
- 一系列的顶点 $V = (v_1, v_2, \dots, v_n)$
- F 中的每个面片是 V 中顶点的序列组 e.g.
 - $f_1 = (v_1, v_2, v_3)$,
 - $f_2 = (v_4, v_5, v_6)$,
 - $f_3 = (v_7, v_8, v_9)$,
 - ...

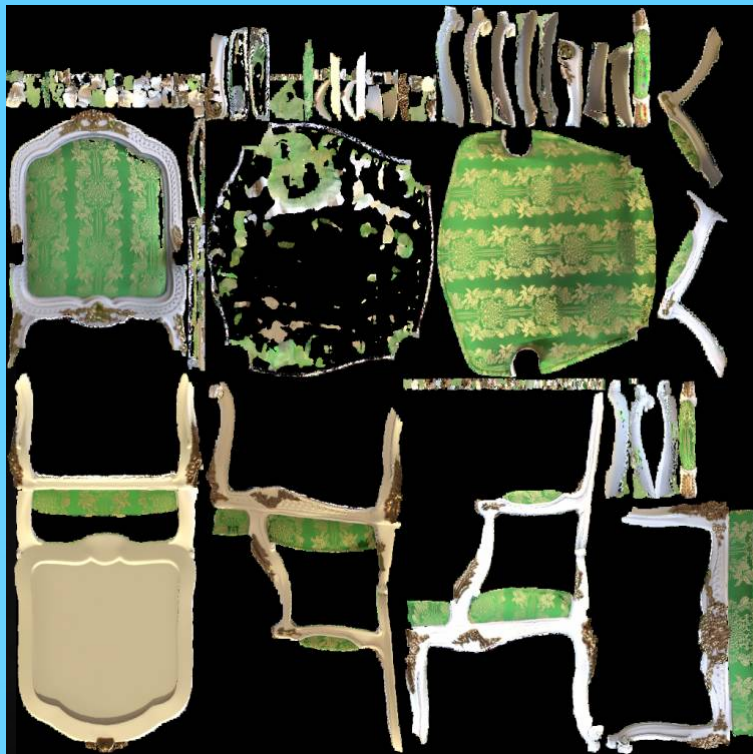


几何表征方法 – 网格与纹理贴图



几何

+



纹理图

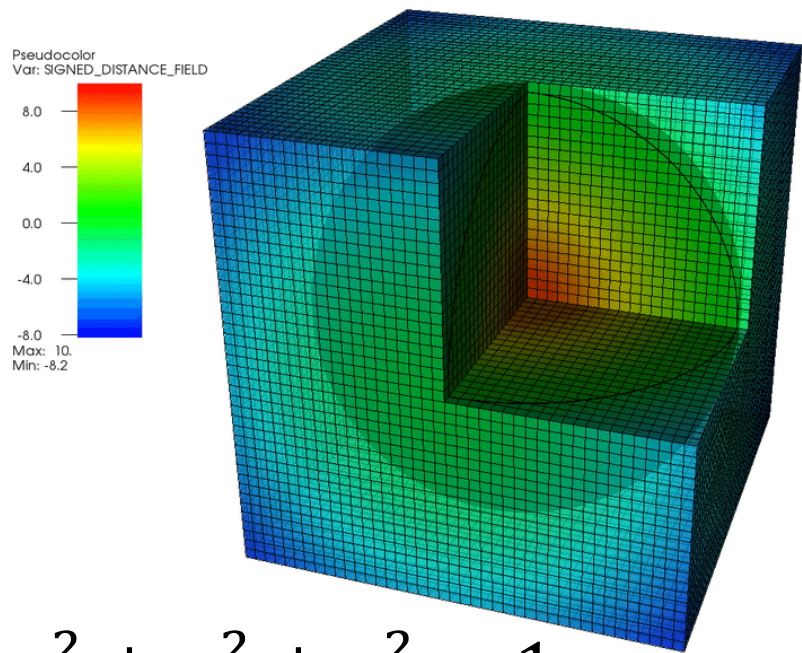
=



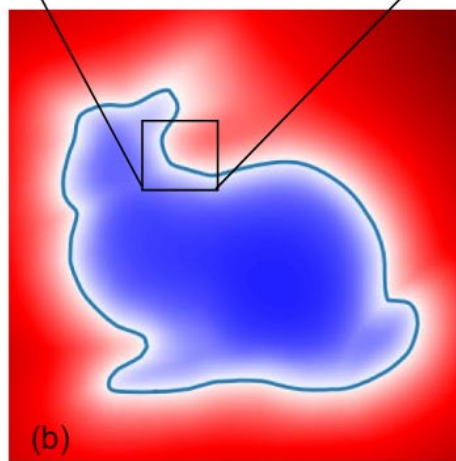
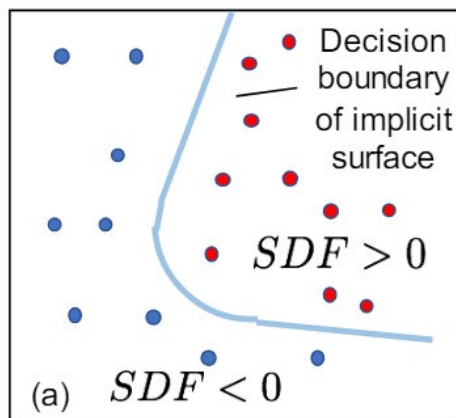
几何表征方法 – 隐式场

- 三维空间点的坐标到SDF值的映射，即每个三维点 x 都对应一个SDF值 s 。

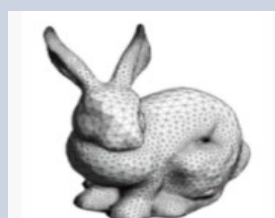
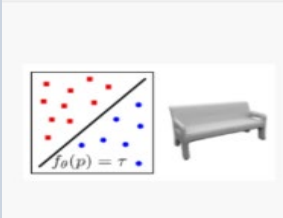
$$SDF(x) = s: x \in \mathbb{R}^3, s \in \mathbb{R}$$



$$x^2 + y^2 + z^2 = 1$$



几何表征方法-总结

表示方法	Voxel	Point Cloud	Polygon Mesh	Occupancy Function	Signed Distance Function
表示图像					
表示原理	体素用规则的立方体表示3D物体, 体素是数据在三维空间中的最小分割单位,类似于2D图像中的像素	点云将多面体表示为三维空间中点的集合,一般用激光雷达或深度相机扫描后得到点云数据,	多边形网格将多面体表示为顶点与面片的集合, 包含了物体表面的拓扑信息	occupancy function将物体表示为一个占有函数, 即空间中每个点是否在表面上	SDF将物体表示为符号距离函数, 即空间中每个点距离表面的距离
优缺点	+规则表示,容易送入网络学习 +可以处理任意拓扑结构 -随着分辨率增加,内存呈立方级增长 -物体表示不够精细 -纹理不友好	+容易获取 +可以处理任意拓扑结构 -缺少点与点之间连接关系 -物体表示不够精细 -纹理不友好	+高质量描述3D几何结构 +内存占有较少+纹理友好 -不同物体类别需要不同的mesh模版 -网格较难学习	+可以精细建模细节,理论上分辨率无穷 +内存占有少 +网络较易学习 -需后处理得到显式几何结构	+可以精细建模细节,理论上分辨率无穷 +内存占有少 +网络较易学习 -需后处理得到显式几何结构

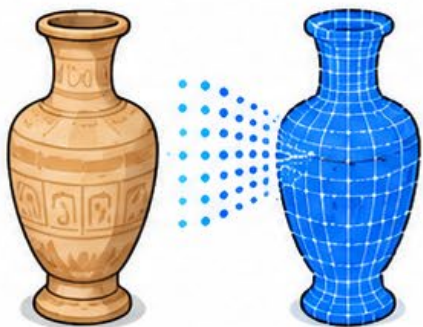


三维重建：3D扫描

3D扫描可以把真实世界快速变成数字资源，
方便保存、分析、展示和再创造。

1 文物与博物馆

为文物建档、修复和
数字展示。



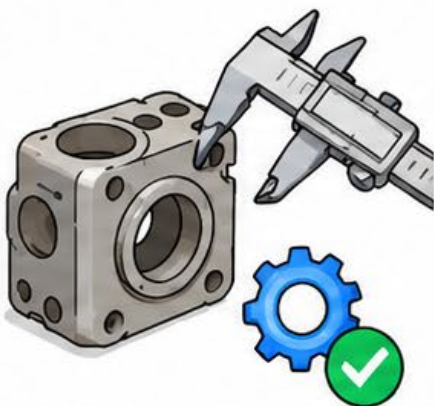
2 电影与游戏

把道具、人物或场景
变成3D素材。



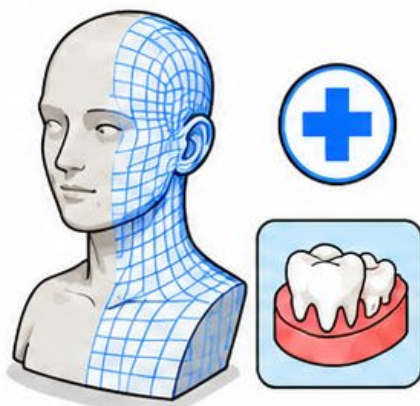
3 工业设计

用于零件测量、逆向
建模和质检。



4 医疗与人体

用于人体建模、牙科
和康复分析。



5 电商与教育

做3D商品展示，也能
用于教学演示。



一句话理解：3D扫描让真实物体进入数字世界

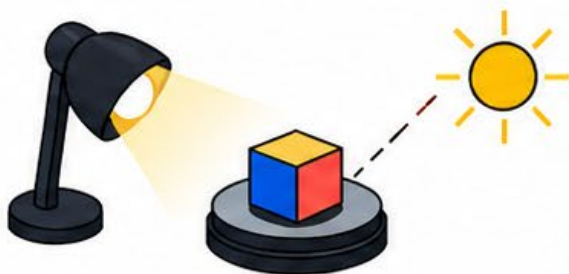


什么会影响扫描效果？

想扫得清楚，不只是设备要好，环境、物体和操作方式都会影响结果。

光照条件

光线太暗或反光太强，都会影响识别。



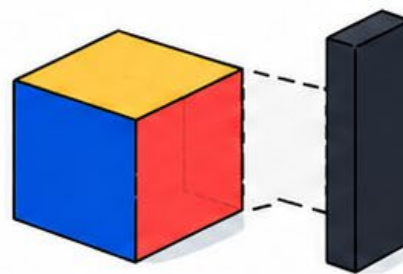
表面材质

透明、反光、纯黑表面最难扫描。



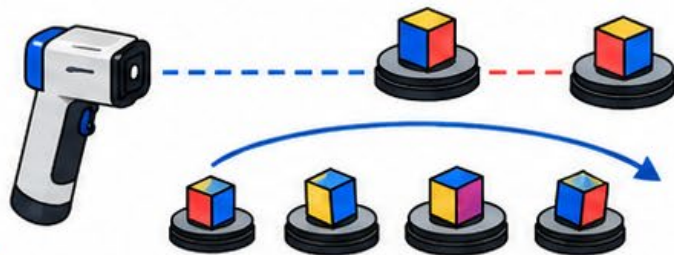
遮挡与死角

看不到的地方，数据就容易缺失。



距离与角度

太远太近，或角度单一，效果都会变差。



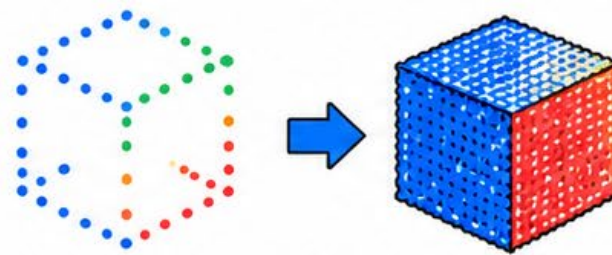
物体是否运动

扫描时要尽量保持稳定，减少抖动。



分辨率与精度

采样越细，细节越多，但速度更慢。





AI时代，如何更快创建3D模型？

如果没有3D扫描仪，能不能只用相机拍到的照片或视频，重建出3D模型？

从设备扫描 → 图像重建

1 传统方式：3D扫描

需要专门设备，成本较高，流程也更复杂



专用设备



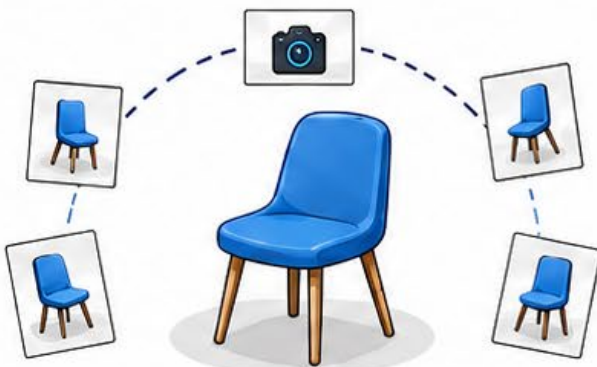
逐步扫描



成本更高

2 新问题：只用相机行不行？

把多张照片 / 一段视频，变成对物体的观察数据



普通相机



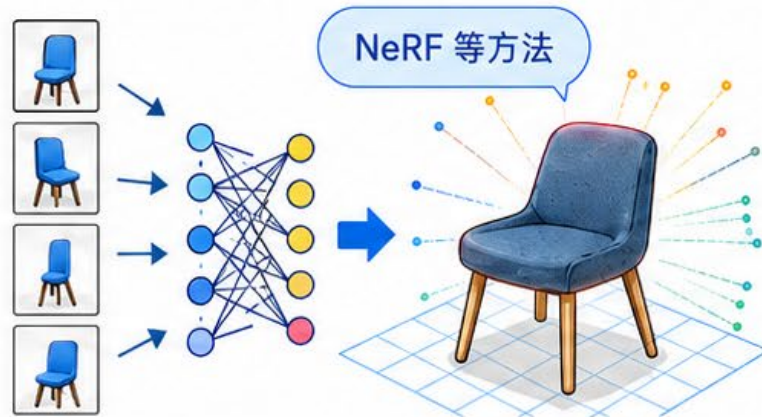
多视角图像



更容易获取

3 AI方法：从图像重建3D

让计算机根据照片推断形状、颜色和视角



更快建模



更灵活



适合数字内容生成

AI重建透明物体

Scene: GlassIce

玻璃杯，吸管和冰块

Scene: BallStatue

*Scene Captured **in the Wild** Using a Mobile Phone

雕像

Scene: Bottle

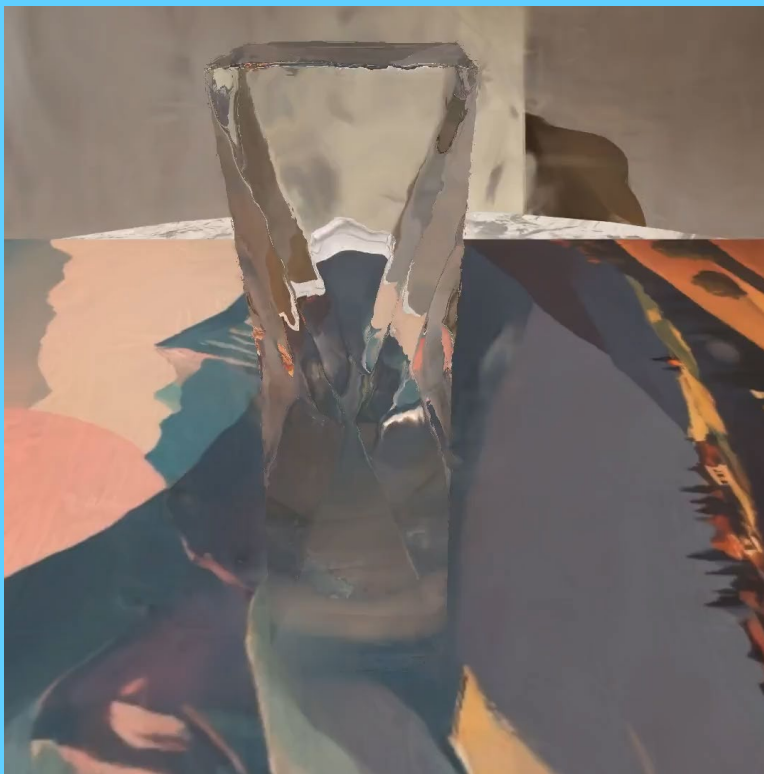
*Scene Captured **in the Wild** Using a Mobile Phone

装水的矿泉水瓶

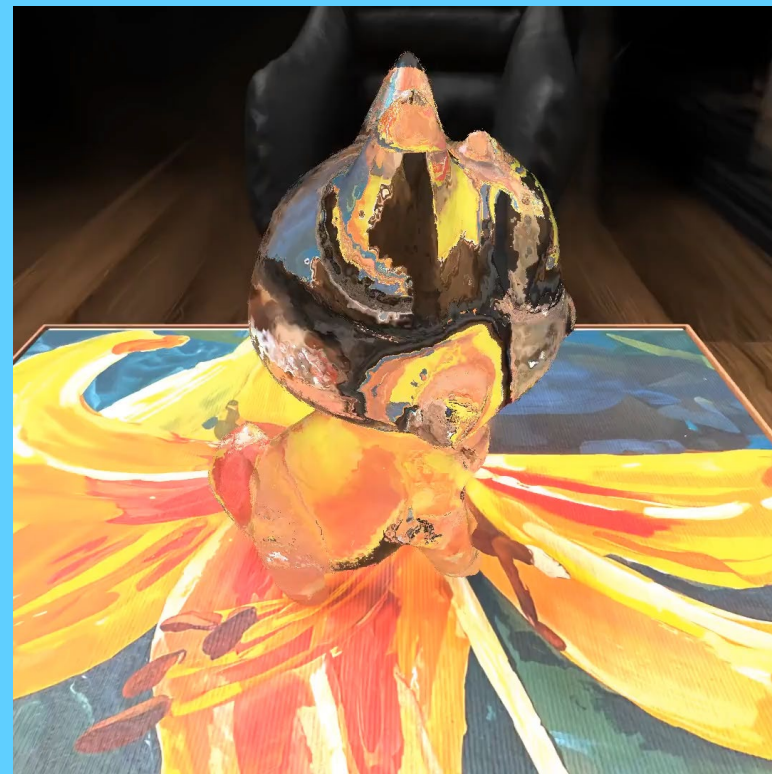
AI重建透明物体



水晶模型



水晶模型



水晶猫

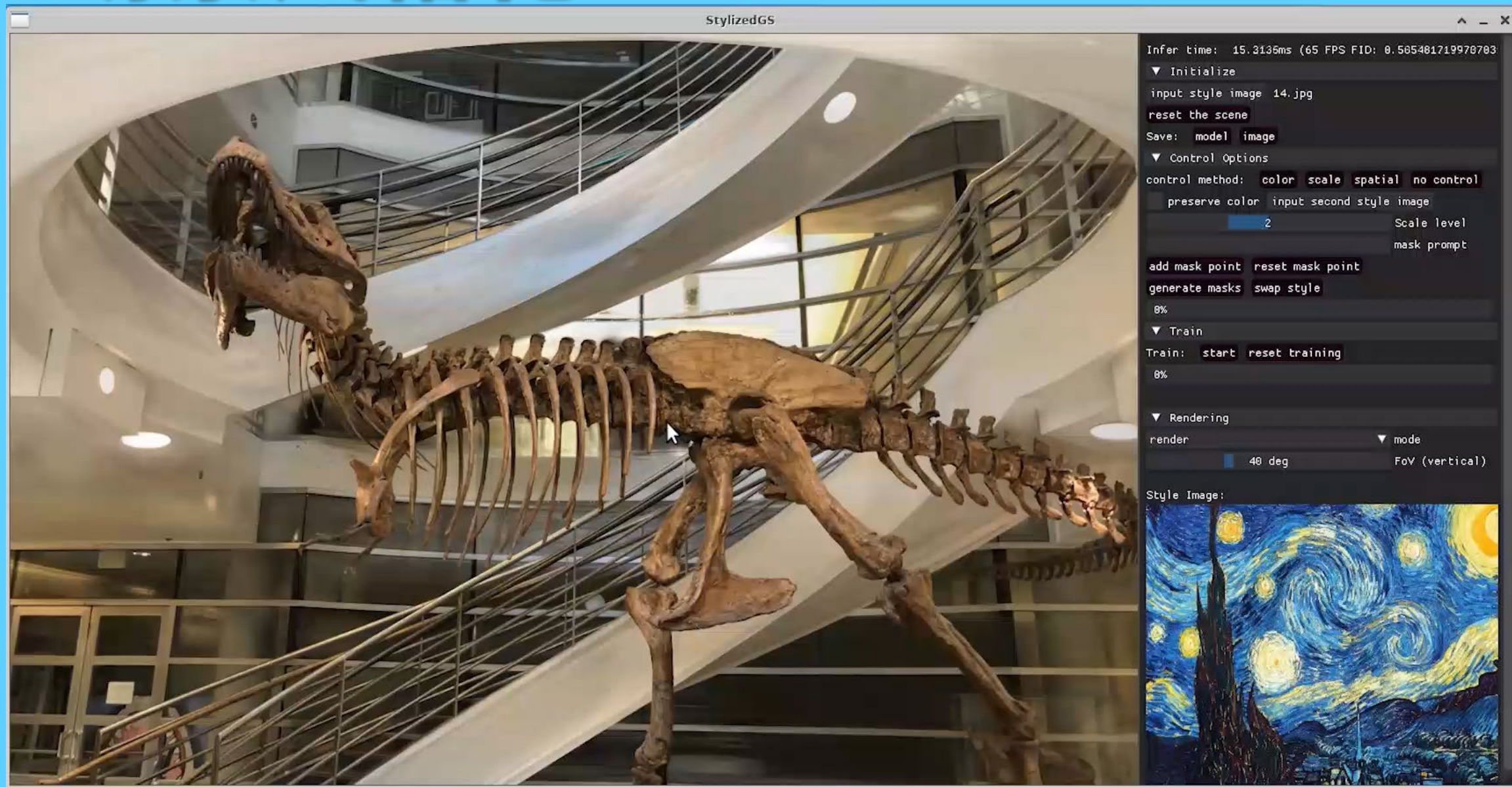
AI重建真实场景



AI物体编辑

Real Time Visualization
on RTX 3090 24GB

AI场景风格化



AI视频编辑

Editing



Input Video

AI视频编辑

Inpainting



Input Video

AI视频编辑

Generation

Input Image



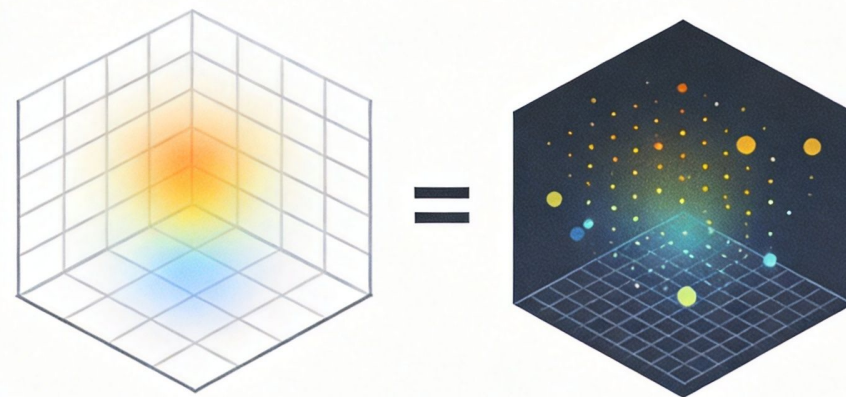
Camera Trajectory

NeRF: 神经辐射场



把3D空间变成看不见的彩色云朵

每个点存储颜色+透明度，
AI学习后可以任意角度渲染



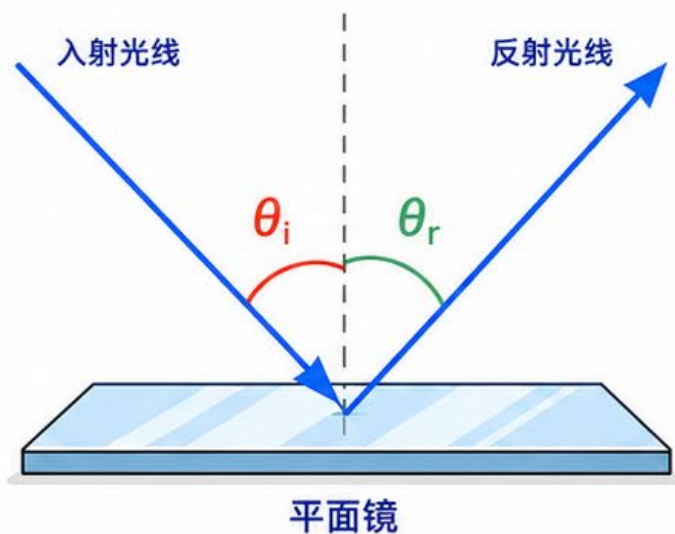
输入照片 → AI学习 → 任意视角3D渲染

光线基本原理（一）：反射与折射

在真实世界里，光线遇到不同表面时，可能会反弹回来，也可能会改变方向继续前进。

1 反射

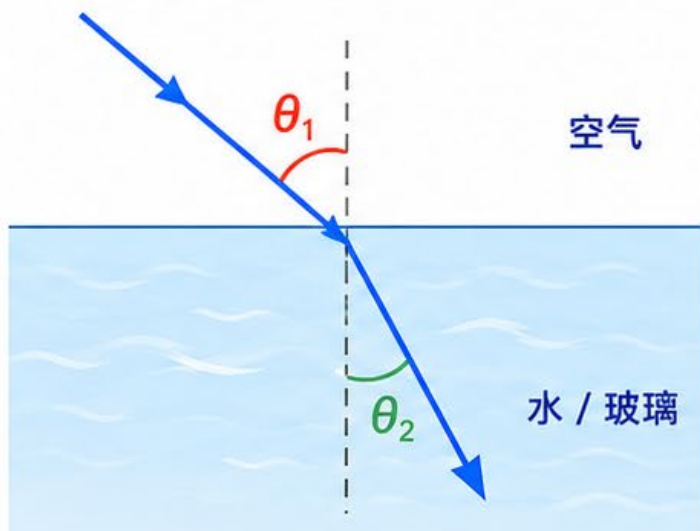
光碰到表面后，会被弹回去。



入射角 = 反射角

2 折射

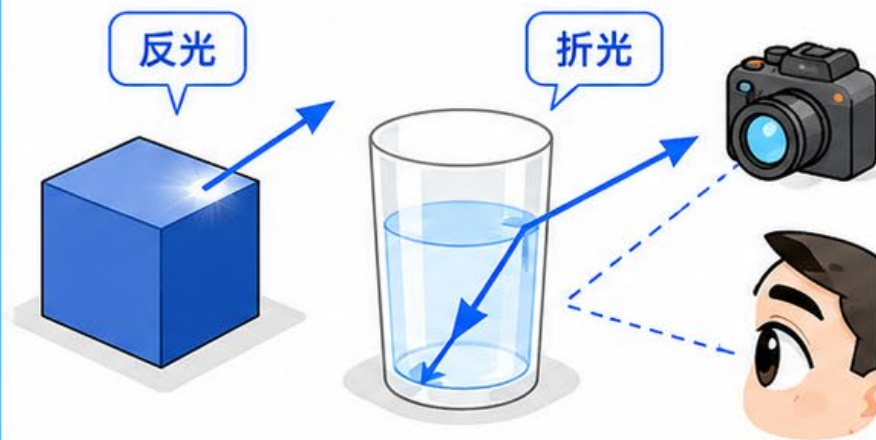
光从一种介质进入另一种介质时，方向会改变。



速度变化 → 方向变化

3 为什么重要？

我们看到的颜色和形状，和光线怎么走有关系。



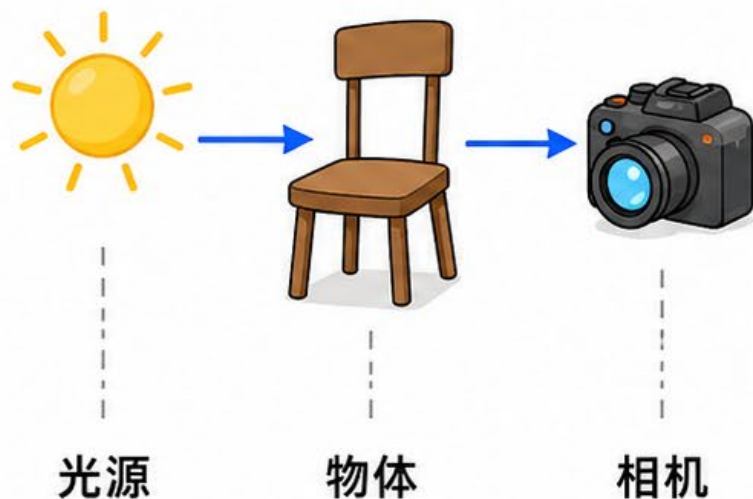
理解反射和折射，才能进一步理解渲染与 NeRF。

光线基本原理（二）：光路可逆

如果光能从物体到相机，那么沿同一条路线反过来走，从相机追踪到物体，结果也是等价的。

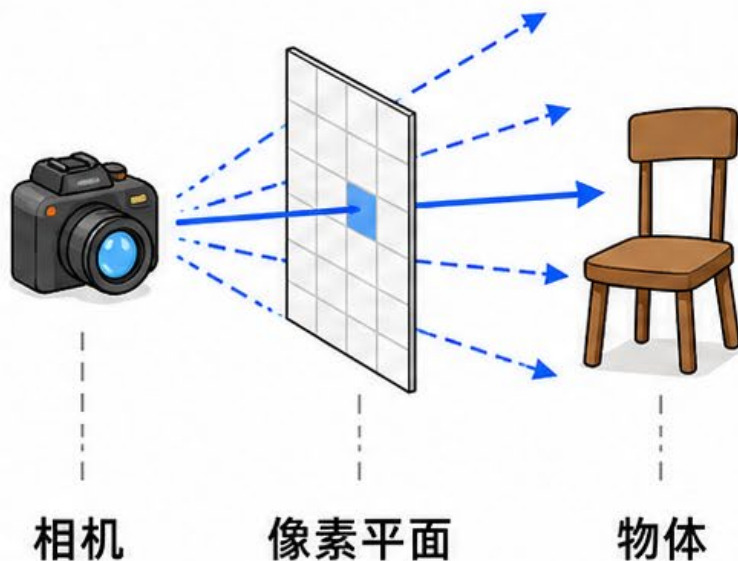
1 真实世界中的光

光通常从光源出发，经过物体，最后进入眼睛或相机。



2 计算机里的做法

为了方便计算，我们常常从相机反向发射光线，去寻找它看到了谁。



3 为什么这样可行？

因为路径还是同一条，只是方向反过来，所以可以得到相同的可见结果。



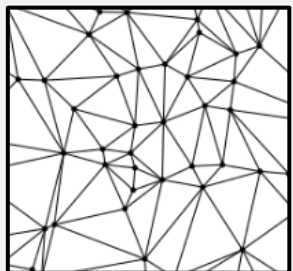
★ 这就是“光路可逆”

经典几何表征的局限性

经典图形 : **几何** + **物理**

神经图形 : 几何 + 物理 + 数据 (神经网络)

离散空间表示



顶点面片



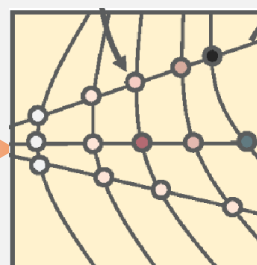
光照



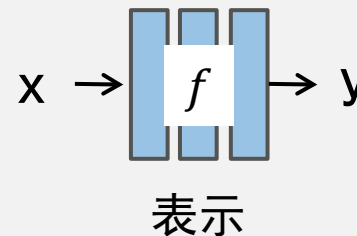
纹理贴图

精度受限 拓扑受限

连续隐式表示

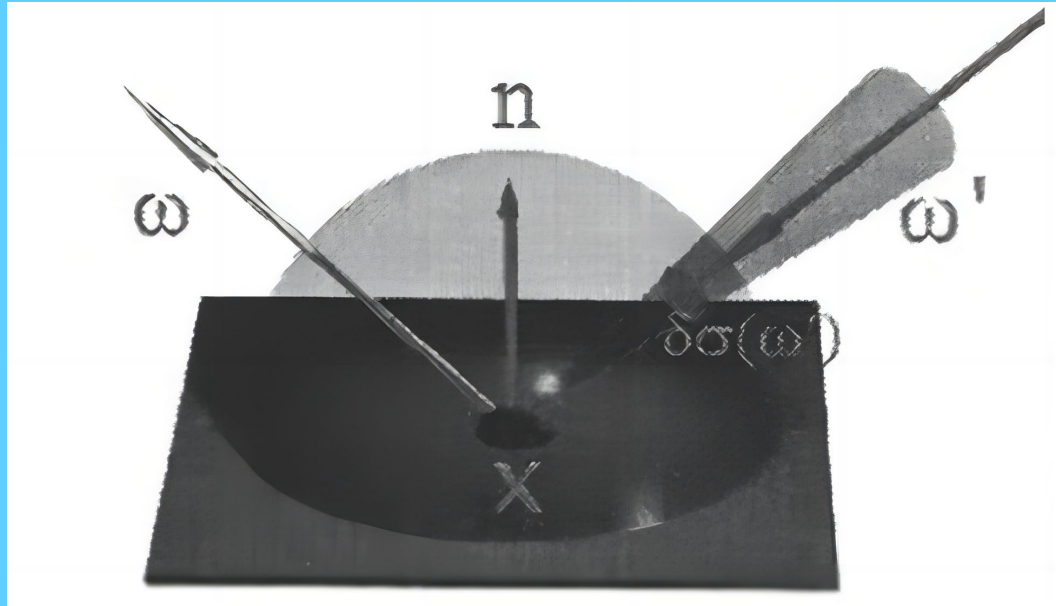


采样点 x
属性值 y



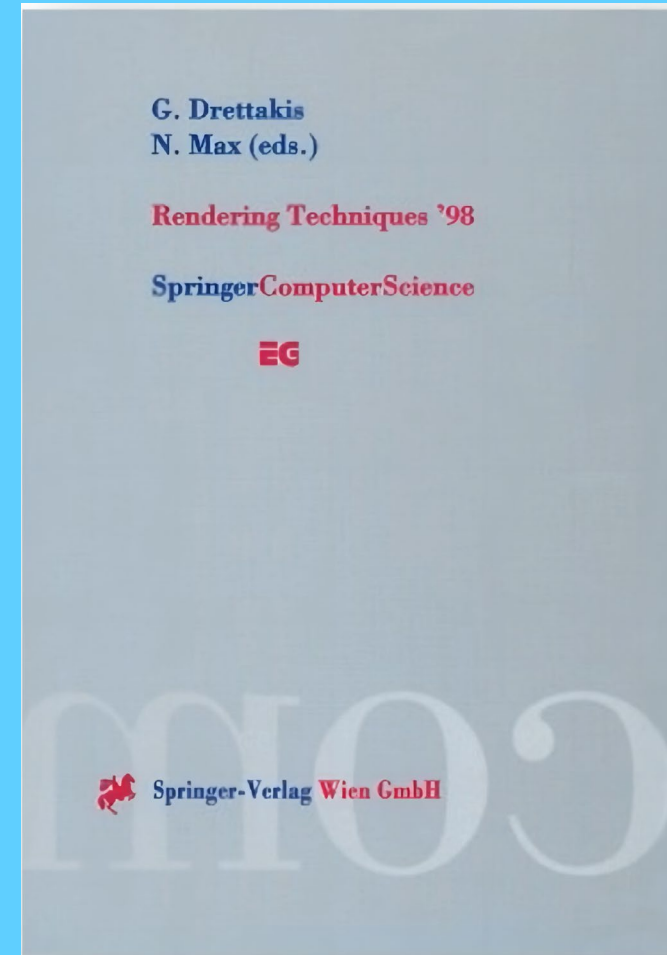
复杂拓扑 表示灵活

通过神经网络近似反射函数



[Gargan and Neelamkavil 1998]

使用神经网络近似反射函数



神经网络基础：多层感知机

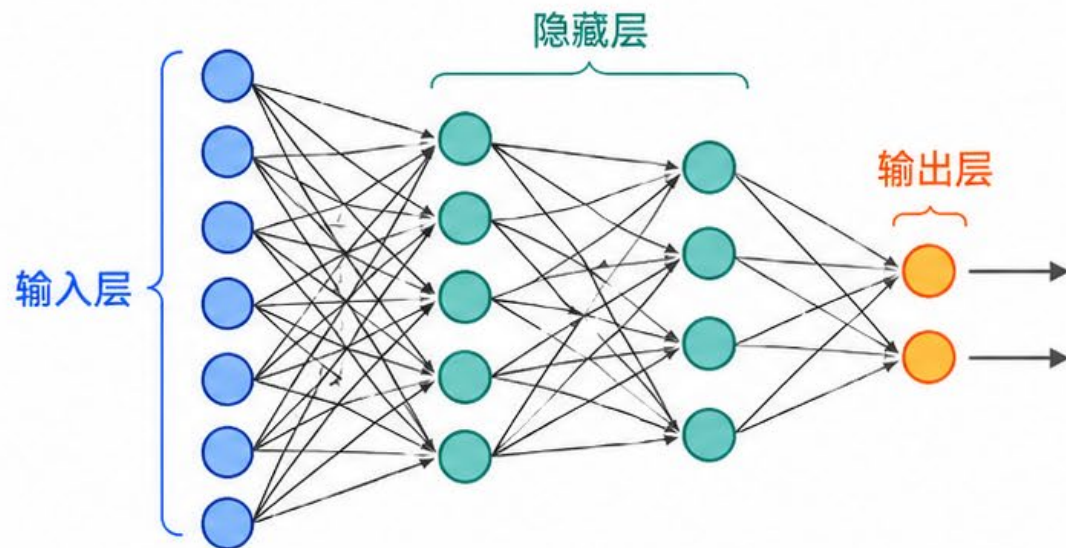
1 多层感知机的基本思想

- 输入排列成列向量，输出中的每一个分量都和输入的每个分量有关
- 由多个全连接层组成，每一个全连接层对应一个矩阵乘法

$$out = in \cdot W^T$$

$$out: (N, L_{out}), W: (L_{out}, L_{in}), in: (N, L_{in})$$

W 为可训练的网络参数。



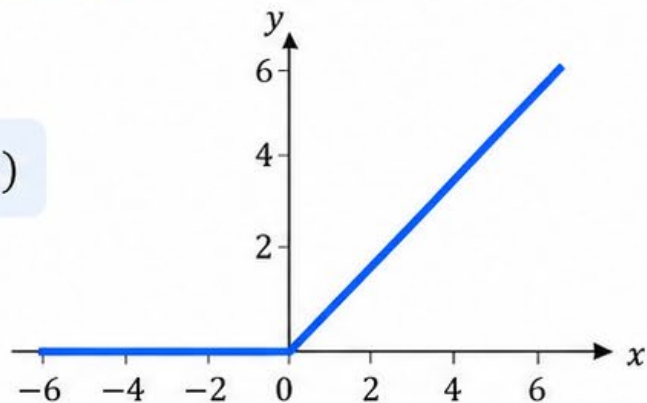
2 常用的两个激活函数

2.1 ReLU

$$ReLU(x) = \max(0, x)$$



优势：缓解
梯度消失问题

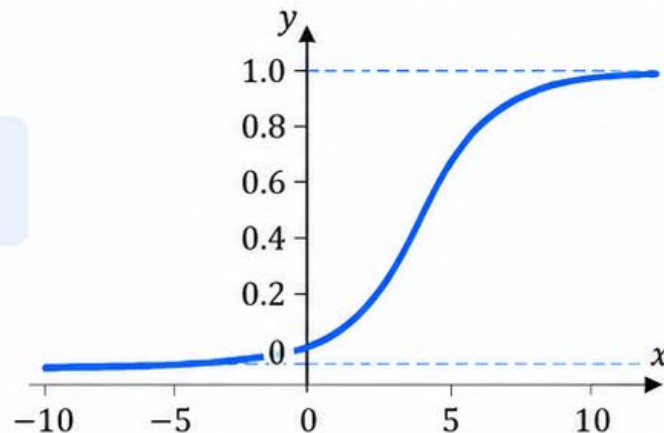


2.2 Sigmoid

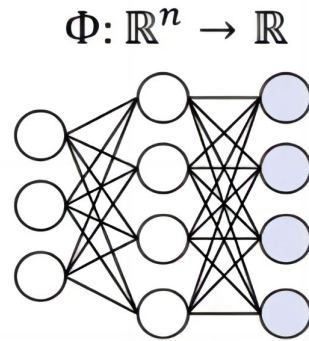
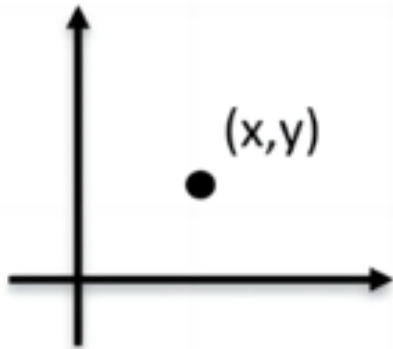
$$Sigmoid(x) = \frac{1}{1 + e^{-x}}$$



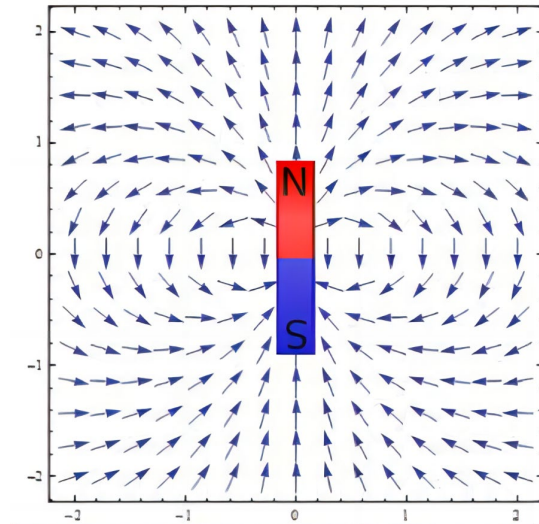
优势：输出范围
在 0 到 1 之间



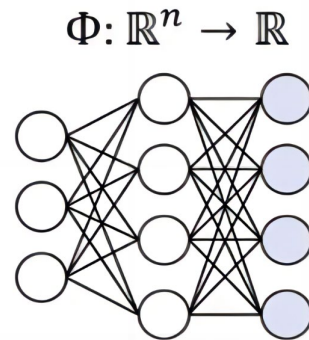
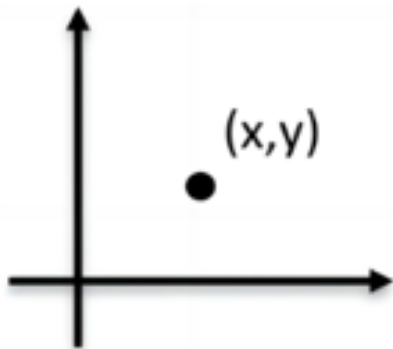
神经辐射场的基本原理



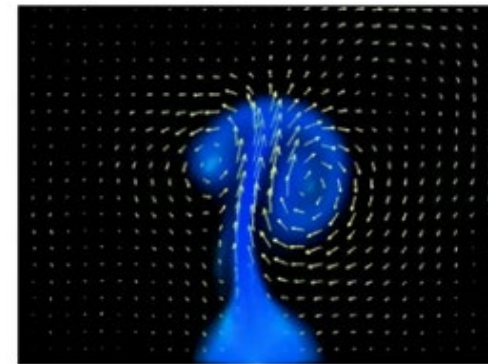
神经网络 (Φ)



Magnetic Field 磁场

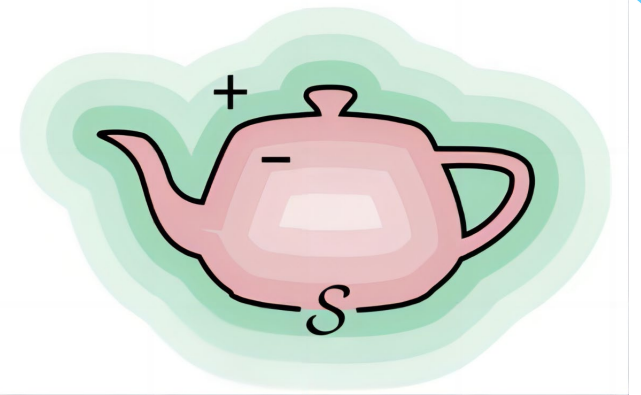
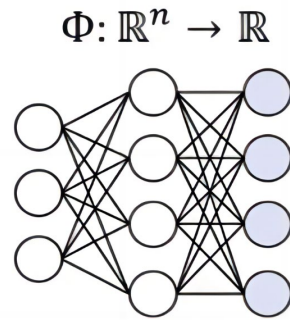
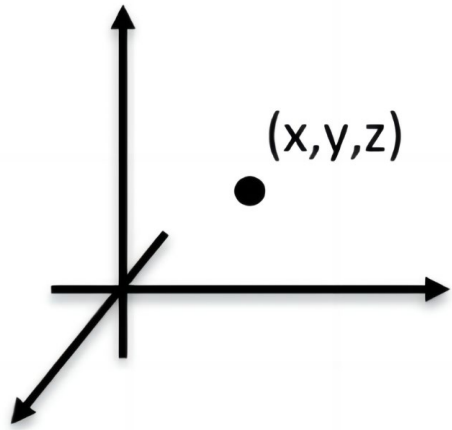


神经网络 (Φ)



Eulerian Flow Field of a Fluid
[Koldora CC]

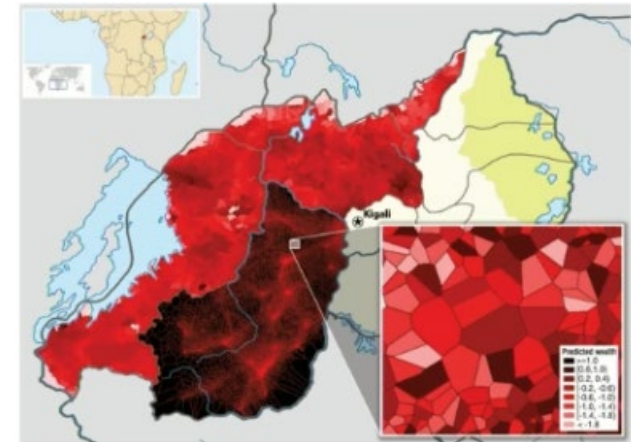
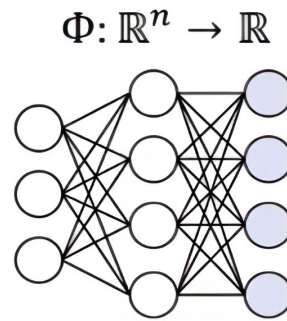
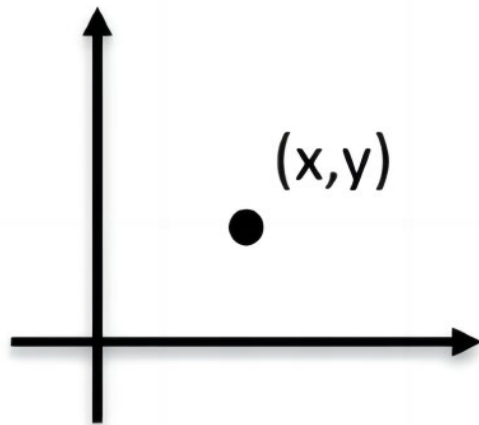
神经辐射场的基本原理



符号距离函数

Signed Distance Function (SDF)

神经网络 (Φ)

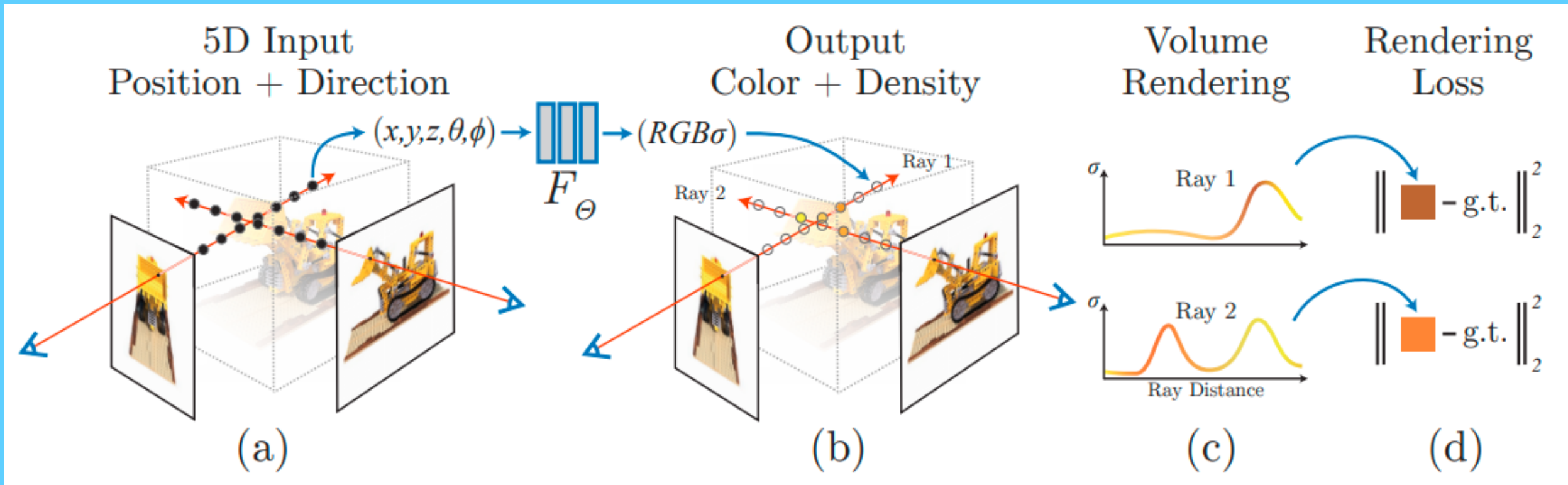


地理空间数据 Geospatial Data
[Blumenstock et al. 2015]

神经网络 (Φ)

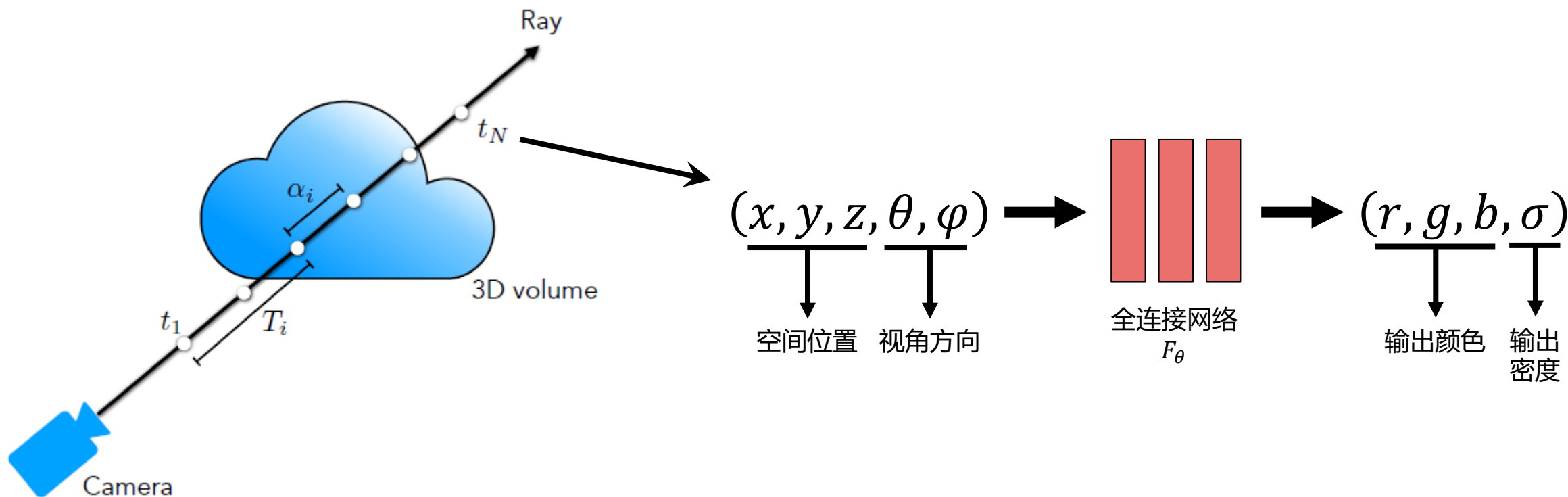
神经辐射场的基本原理

- 使用全连接网络建模5D全光函数，使用体渲染得到最终的任意视角图像



神经辐射场的基本原理

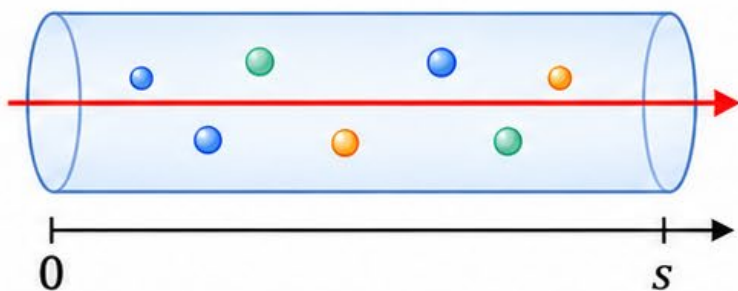
- 基于ray marching: 从相机出发, 向空间中投射光线, 并在光线上采样。
- 每个采样点送入全连接网络, 预测颜色和密度值, 整个过程是可微的。



体渲染（一）：光为什么会变弱？

想像一束光沿着一条射线往前走。路上如果有很多小颗粒，光就会被一点点挡住，所以会越来越弱。

1 光线沿射线传播



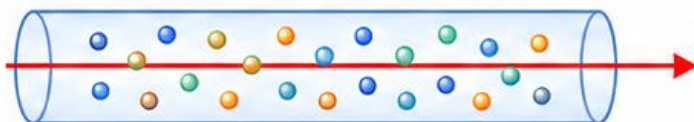
射线上不同位置，可以有不同多少的粒子。

2 粒子越多，衰减越强

粒子少的区域（密度小）



粒子多的区域（密度大）



用 $\sigma(s)$ 表示该位置附近的“密度”。
 $\sigma(s)$ 越大，光被削弱得越快。

3 一小段路上的变化

$$I(s + \Delta s) = I(s) - \sigma(s)I(s)\Delta s$$

- $I(s)$: 走到 s 时的光强
- Δs : 继续前进的一小段距离
- $\sigma(s)$: 这一小段里的密度

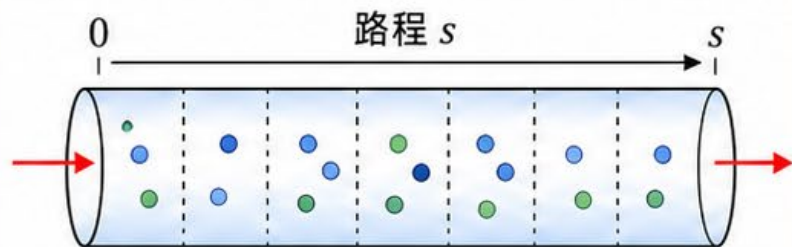
意思：这一小段里，密度越大，
原本光越强，损失就越多。

一句话理解：体渲染先研究——光沿着一条射线前进时，是怎样一点点变弱的。

体渲染（二）：什么是透明度 $T(s)$ ？

如果每一小段路都会让光变弱，那么走完整段路以后，还剩下多少光？这就是透明度（透射率）的意思。

1 把很多小段连起来



前一页讲的是：每一小段都会削弱一点光。
把这些小变化连续叠加起来，
就得到整段路上的结果。

2 透明度 $T(s)$

$$T(s) = e^{-\int_0^s \sigma(t) dt}$$

$$I(s) = I(0)T(s)$$

- $T(s)$ ：光走到 s 位置后，还剩下的比例
- $T(s)$ 越小，说明挡住光的粒子越多
- $T(s)$ 越大，说明这条路更“透明”

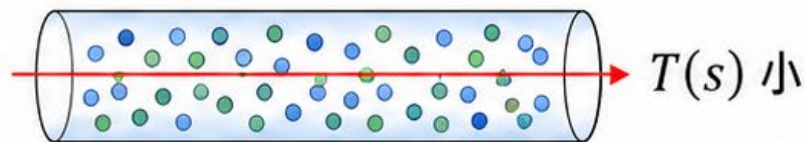
透明  不透明

3 怎么直观理解？

薄雾 / 粒子少



浓雾 / 粒子多



意思：透明度就是“这条路有多容易让光通过”。

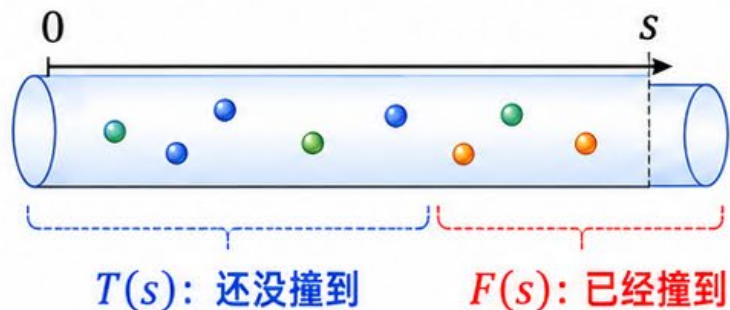
一句话理解：体渲染里， $T(s)$ 表示光走到某个位置时还能剩下多少，也可以理解成这一路有多透明。

体渲染（三）：碰撞概率和颜色怎么累加？

前面我们知道了透明度 $T(s)$ 表示光走到某个位置还剩多少。接下来要问：光最可能在什么位置碰到粒子？碰到后颜色又怎么累加？

1 什么是 $F(s)$ ？

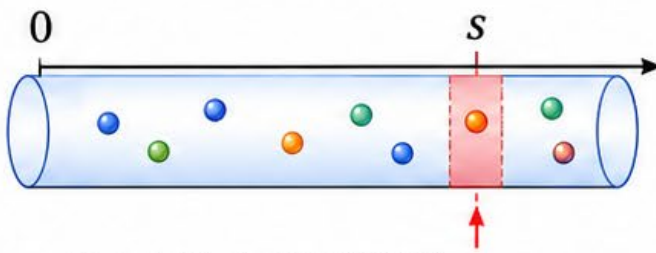
$F(s) = 1 - T(s)$ 表示：光线从起点走到位置 s 时，已经撞到过至少一个粒子的概率。



$$F(s) = 1 - T(s)$$

2 什么是 $F'(s)$ ？

$F'(s)$ 表示：光线在 s 附近“正好撞到粒子”的可能性有多大。

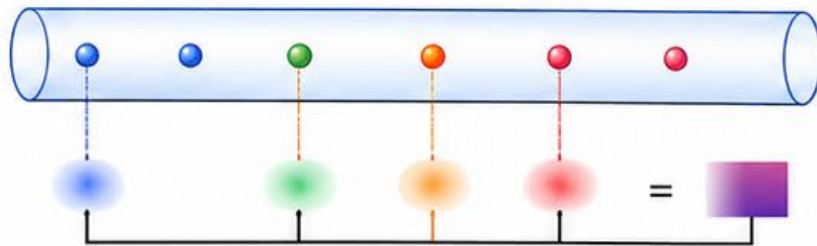


这个位置更可能贡献颜色，
因为这里既“透明 enough” ($T(s)$ 大)，
又“够密” ($\sigma(s)$ 大)。

$$F'(s) = T(s)\sigma(s)$$

3 颜色怎么累加？

如果位置 s 的粒子颜色是 $c(s)$ ，
那么所有位置的贡献要全部加起来，得到最终颜色。



$$C = \int_0^\infty F'(s)c(s)ds = \int_0^\infty T(s)\sigma(s)c(s)ds$$

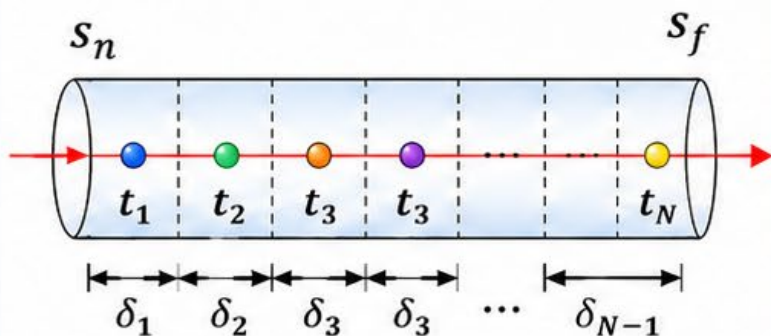
- $T(s)$ ：走到这里还剩多少光
- $\sigma(s)$ ：这里有多“密”
- $c(s)$ ：这里粒子的颜色
- 三者一起决定这一点对最终颜色的贡献

一句话理解：最终颜色 = 把光线上每个位置“撞到粒子的概率 × 该位置颜色”全部加起来。

体渲染（四）：计算机里怎么近似计算？

真正的积分很难直接精确计算，所以计算机会把一条光线切成很多小段，在每一小段里取样，再把这些采样结果加起来。

1 先把光线切成很多小段

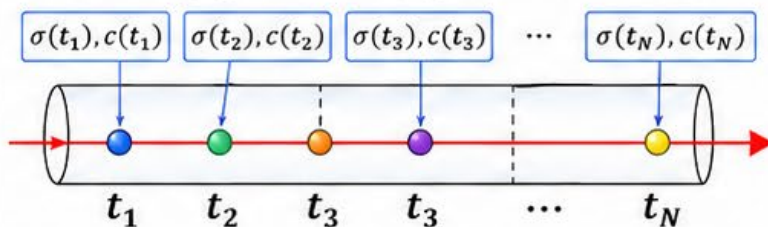


$$\delta_i = t_{i+1} - t_i$$

积分区间改成有限范围 $[s_n, s_f]$ ，
每小段取一个采样点。

2 每个采样点算什么？

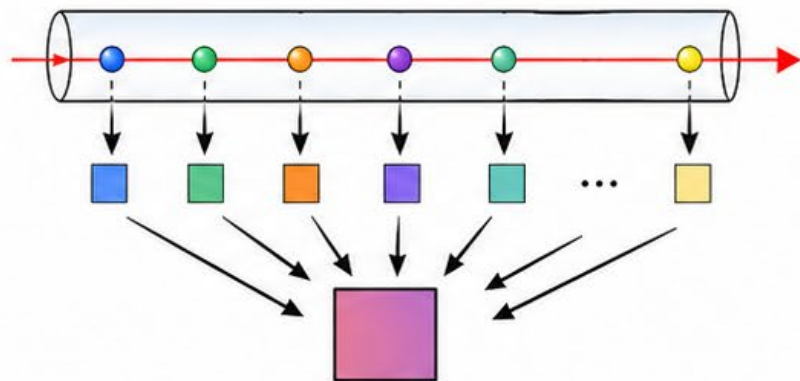
在每个采样点 t_i ，网络会预测该处的密度 $\sigma(t_i)$ 和颜色 $c(t_i)$ 。



$$T(t_i) = \prod_{j=0}^{i-1} e^{-\sigma(t_j)\delta_j}$$

- $\sigma(t_i)$ ：这个点有多“密”
- $c(t_i)$ ：这个点的颜色
- $T(t_i)$ ：光走到这里还剩多少

3 最后把每个点的贡献加起来



$$C \approx \sum_i T(t_i)\sigma(t_i)c(t_i)\delta_i \approx \sum_i w_i c_i$$

w_i 表示第 i 个点的贡献权重。

每个点贡献一点颜色，所有点相加，
就得到这条光线对应像素的颜色。

一句话理解：体渲染就是“沿光线采样很多点 → 计算每个点的贡献 → 全部加起来”。

体渲染（五）：连续积分怎么变成离散求和？

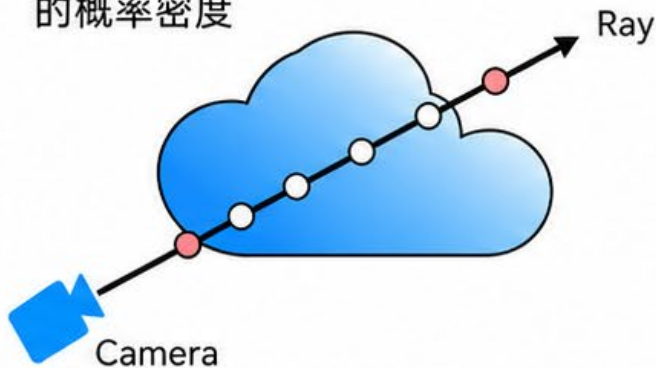


真实公式是连续积分，但计算机不会无限连续地算，所以要把一条光线切成很多小段，用有限个采样点来近似。

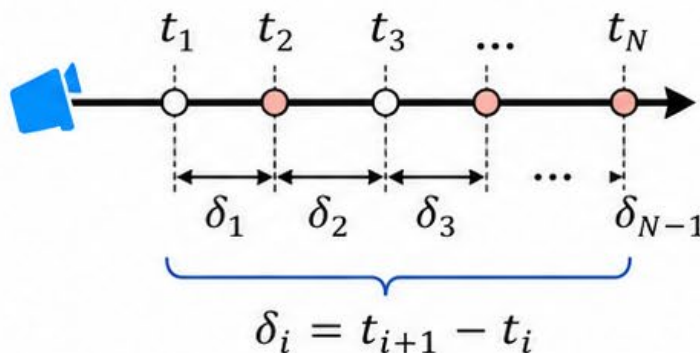
① 先看连续公式

$$C = \int_0^{\infty} F'(s)c(s)ds$$

- $c(s)$: 位置 s 处粒子的颜色
- $F'(s)$: 光线在 s 处“正好碰到粒子”的概率密度



② 把光线切成很多小段



- 积分区间改成有限范围 $[s_n, s_f]$
- 每一小段取一个采样点
- 连续问题 → 离散近似

③ 用差分近似 $F'(s)$

$$F'(t_i) \approx \frac{(1 - T(t_{i+1})) - (1 - T(t_i))}{t_{i+1} - t_i}$$

- 意思：用前后两个位置的概率变化，近似这一小段里的“碰撞可能性”

$$T(t_i) = \prod_{j=0}^{i-1} e^{-\sigma(t_j)\delta_j}$$

- $T(t_i)$: 光到达第 i 个点前还剩多少



一句话理解：体渲染就是把连续积分，变成“沿光线采很多点，再逐点近似计算”。

体渲染（六）：最后的离散公式怎么理解？

💡 把每个采样点的贡献加起来，就能得到这一条光线对应像素的颜色。

① 先把差分代回去

$$C \approx \sum_i ((1 - T(t_{i+1})) - (1 - T(t_i)))c(t_i)$$



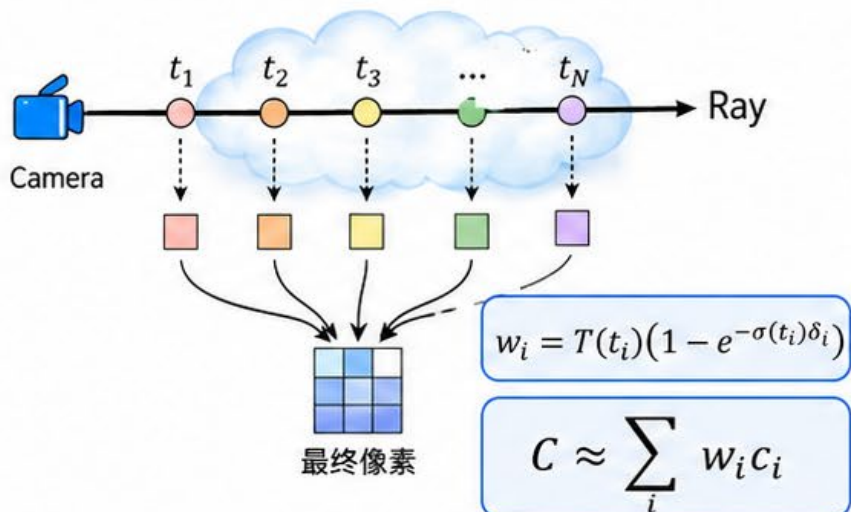
$$= \sum_i (T(t_i) - T(t_{i+1}))c(t_i)$$



$$= \sum_i T(t_i)(1 - e^{-\sigma(t_i)\delta_i})c(t_i)$$

$$T(t_i) = \prod_{j=0}^{i-1} e^{-\sigma(t_j)\delta_j} \quad \delta_i = t_{i+1} - t_i$$

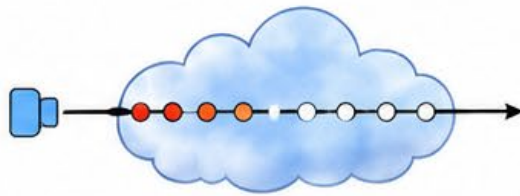
② 每一项都表示什么？



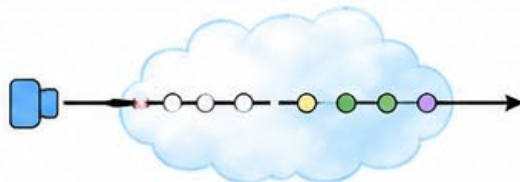
- $T(t_i)$ ：光到这个点时还剩多少
- $1 - e^{-\sigma(t_i)\delta_i}$ ：这一小段里撞到粒子的概率
- c_i ：这个点的颜色
- w_i ：第 i 个点最终颜色的贡献权重

③ 直观理解

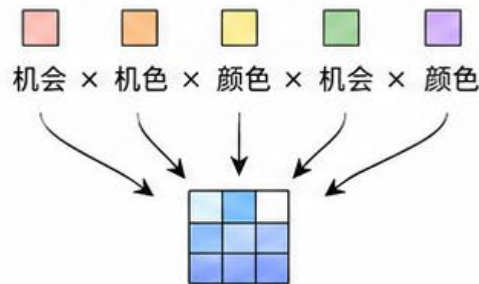
- ① 前面的点如果很密，容易先挡住后面的光



- ② 透明度高的地方，后面的点还有机会贡献颜色



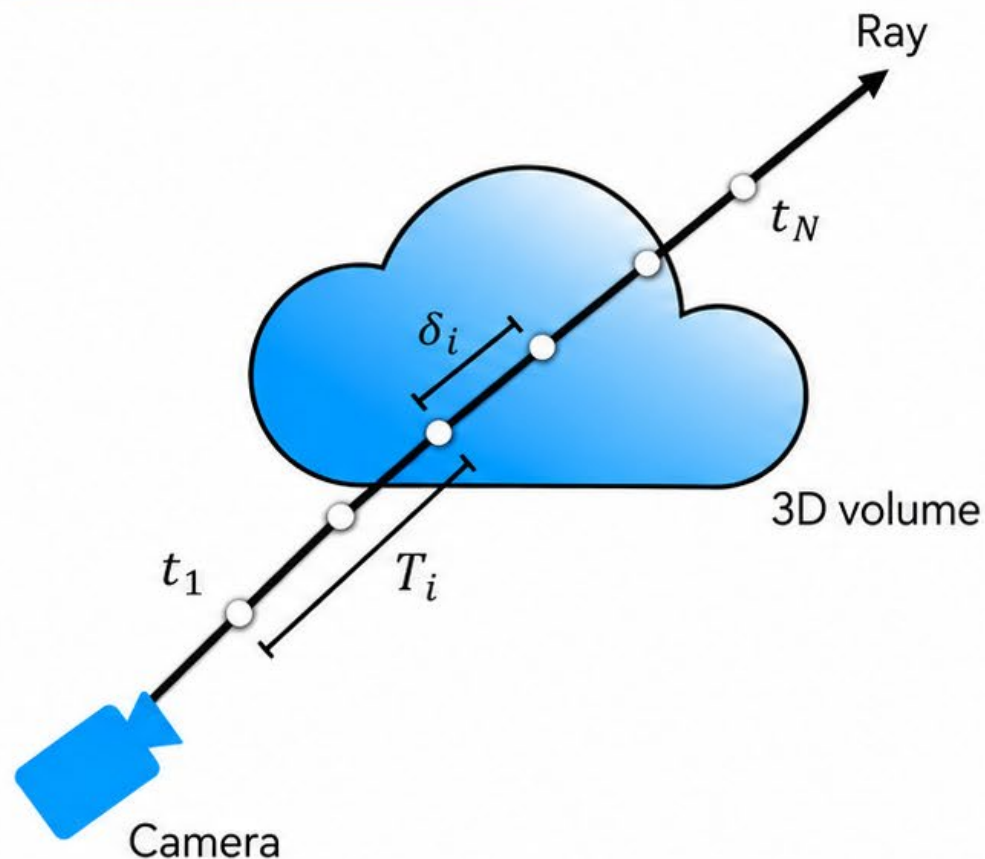
- ③ 最终颜色 = 每个点的“被看到的机会 × 自己的颜色”全部相加



★ 一句话理解：体渲染的离散公式，就是“每个采样点按权重贡献一点颜色，全部加起来”。

体渲染 (七) : 总结

光线穿过体积的示意图



1 颜色计算公式

对于光线 $r(t) = \vec{o} + t\vec{d}$ ，其对应像素的颜色值可以计算为

$$C \approx \sum_{i=1}^N \boxed{T_i \alpha_i} c_i$$

Diagram illustrating the color calculation formula. The formula is $C \approx \sum_{i=1}^N T_i \alpha_i c_i$. A blue box highlights $T_i \alpha_i$, with an arrow pointing to it labeled "weights". An arrow points to c_i labeled "color".

2 透射率 T_i

T_i 衡量了沿着光线有多少能量被提前挡住（未到达第 i 个采样点）

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

3 分段贡献 α_i

α_i 表示第 i 段 ($[t_i, t_{i+1}]$) 对颜色的贡献比例

$$\alpha_i = 1 - e^{-\sigma_i \delta_i}$$

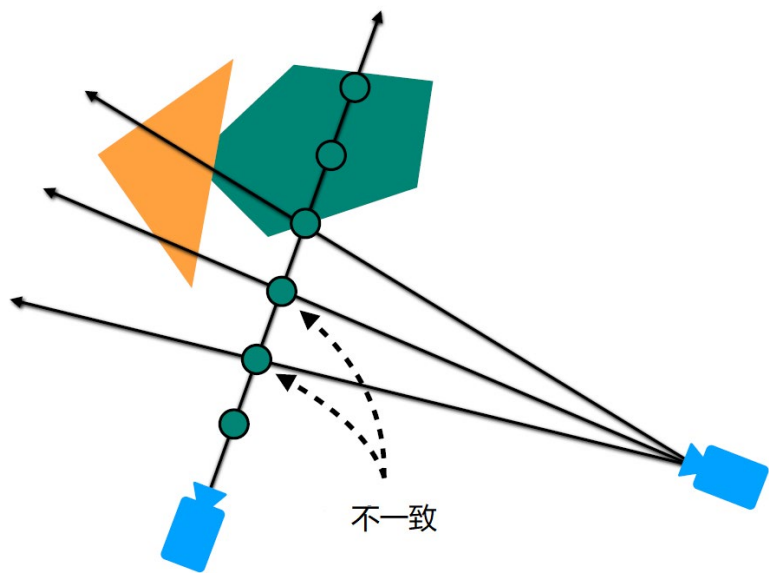


颜色是沿光线的所有采样点贡献的加权和，每个点的权重由透射率 T_i 和分段贡献 α_i 决定。

神经辐射场的基本原理：多视角一致性

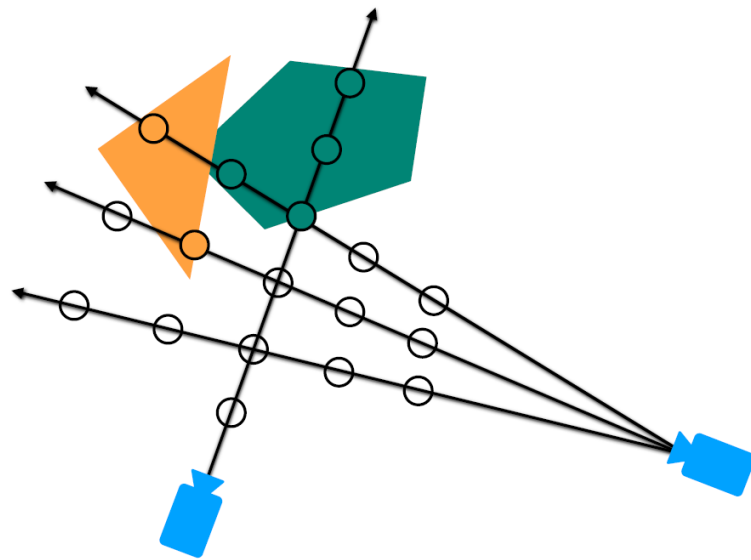
同一个三维点，从不同相机看过去，应该对应一致的颜色和几何位置。NeRF 就是利用这种“多视角一致性”来学习三维场景。

① 不一致会有什么问题？



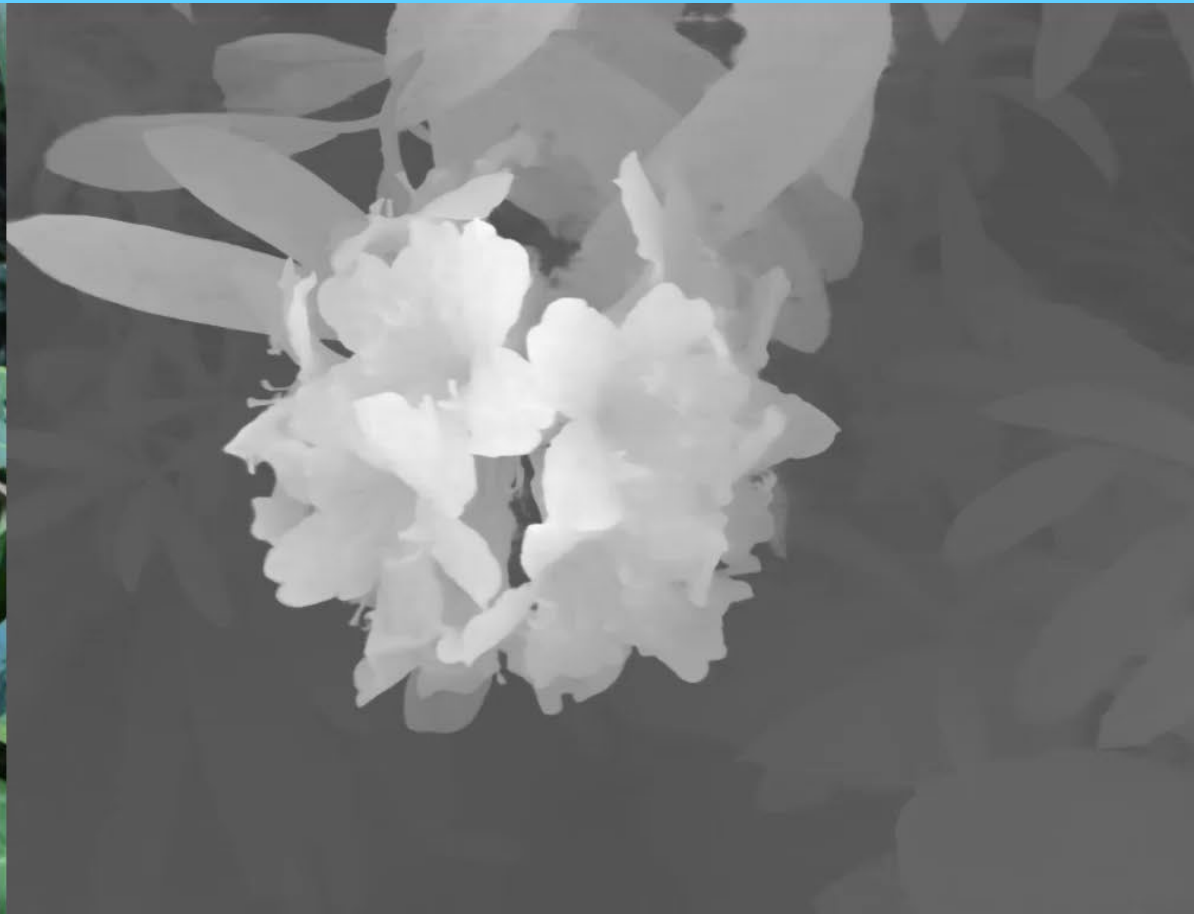
- 不同视角预测出的点对不上
- 颜色和几何会互相矛盾

② 多视角一致监督



- 多个相机一起观察同一场景
- 同一三维点在不同视角下应一致
- 这样能学到稳定的形状和颜色

神经辐射场的结果展示

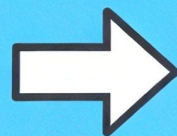


AI生成3D角色

输入：描述性文字

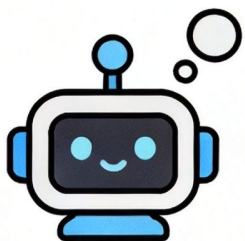
输入框

一只穿盔甲的熊猫

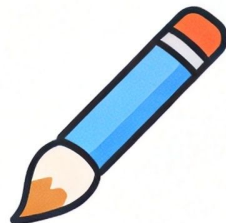


输出：完整的3D角色模型

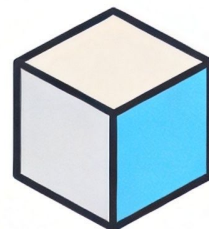
1 理解文字描述



2 生成参考图像



3 从图像重建3D模型



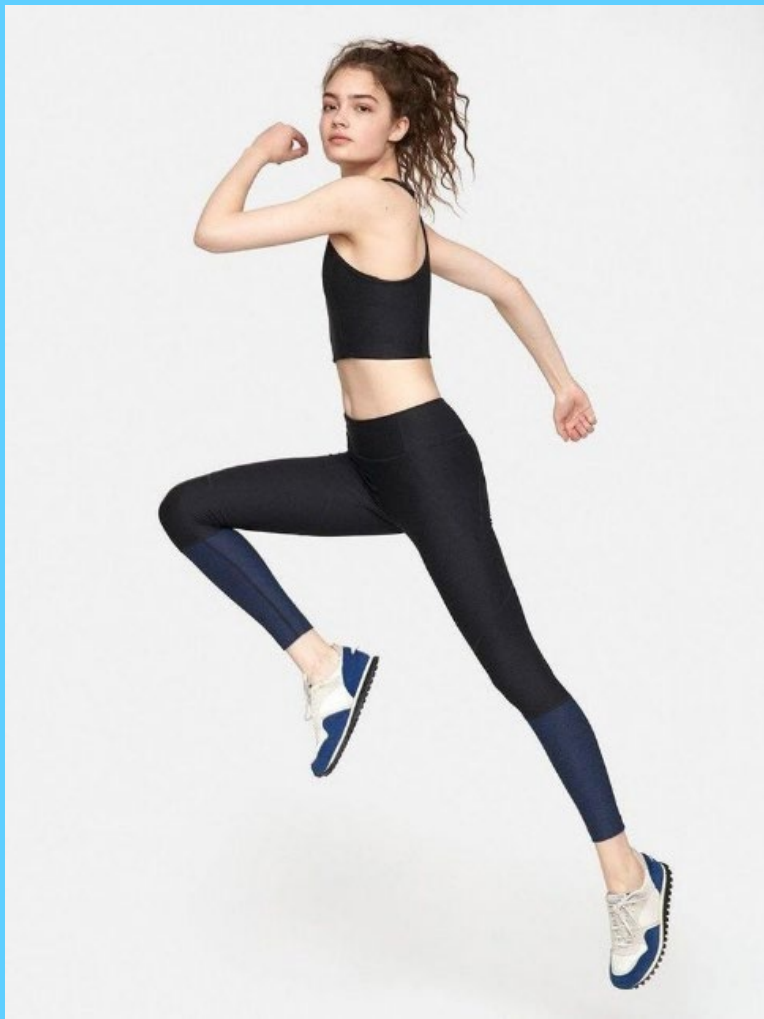
几秒钟就能生成，
以前要几天！



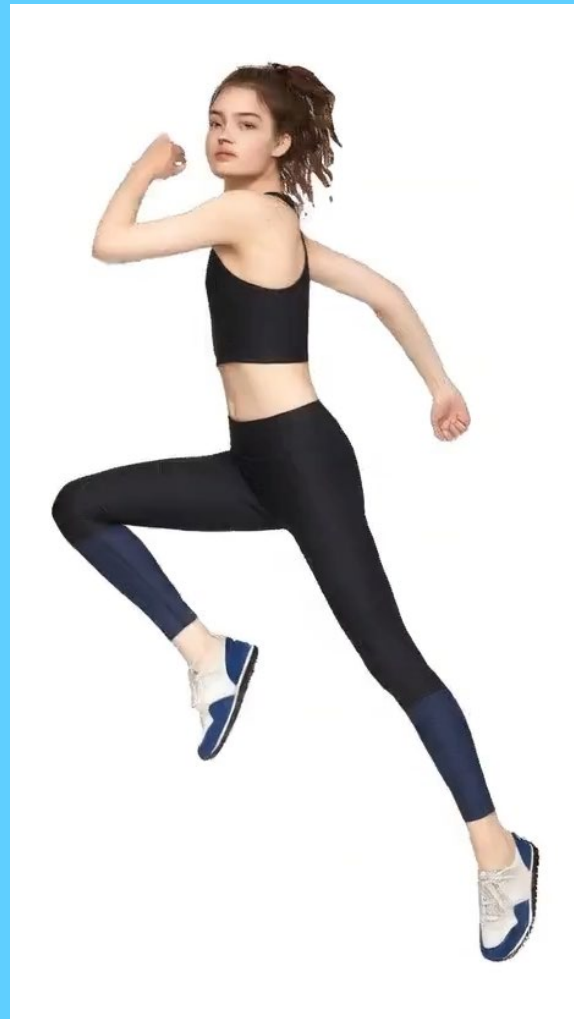
每个人都能拥有 专属数字人

游戏技术走进日常生活

单张图像合成数字人



单张图片



三维数字人

单张图像合成数字人



单张图片



三维数字人

单张图像合成数字人

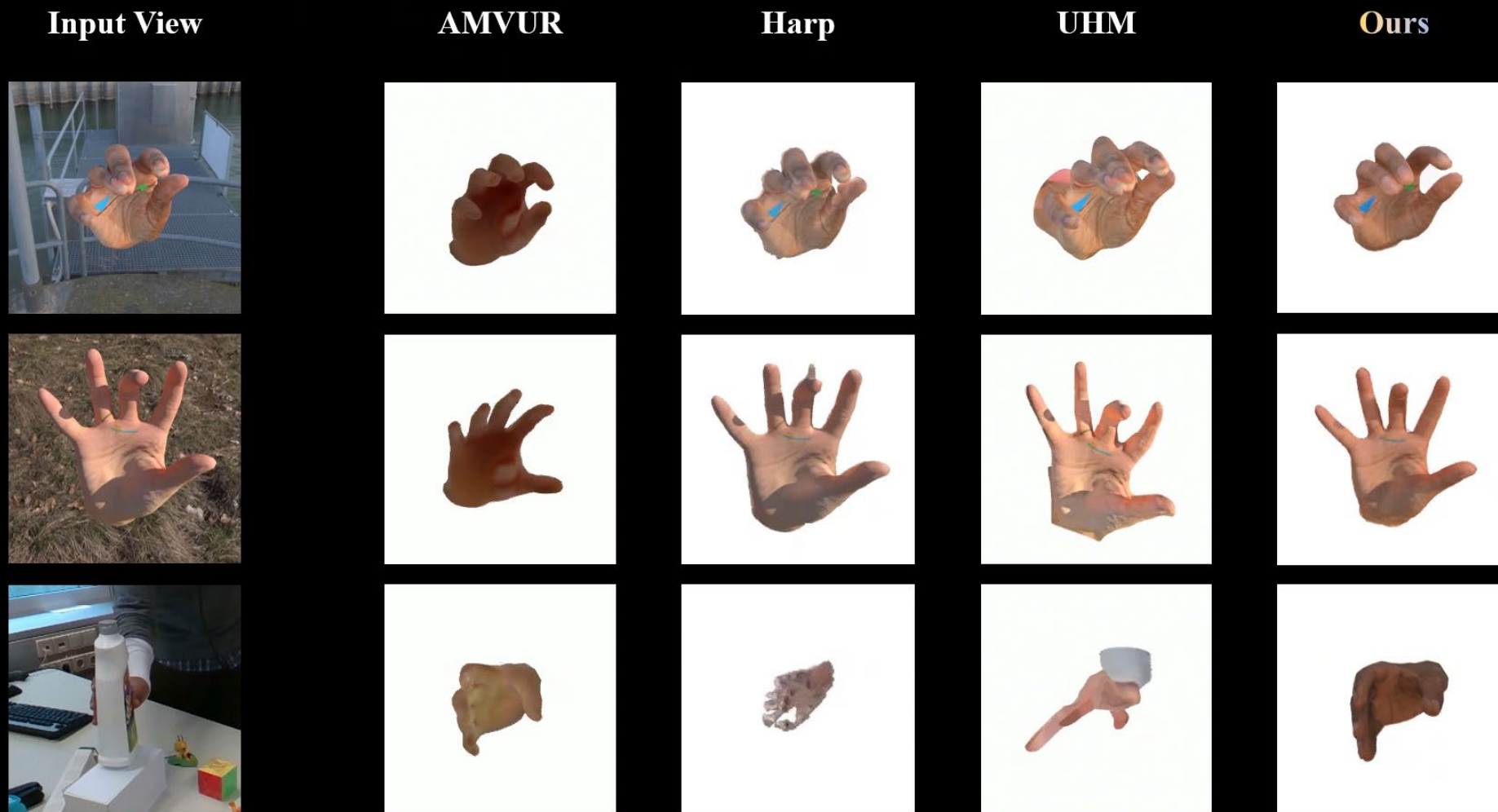


Driving Video

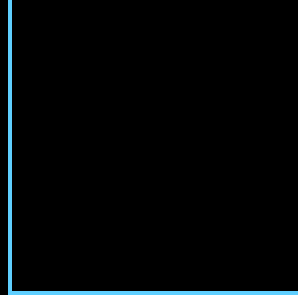


单张图像合成数字人手

Novel View Synthesis



数字人光照编辑



数字人光照编辑

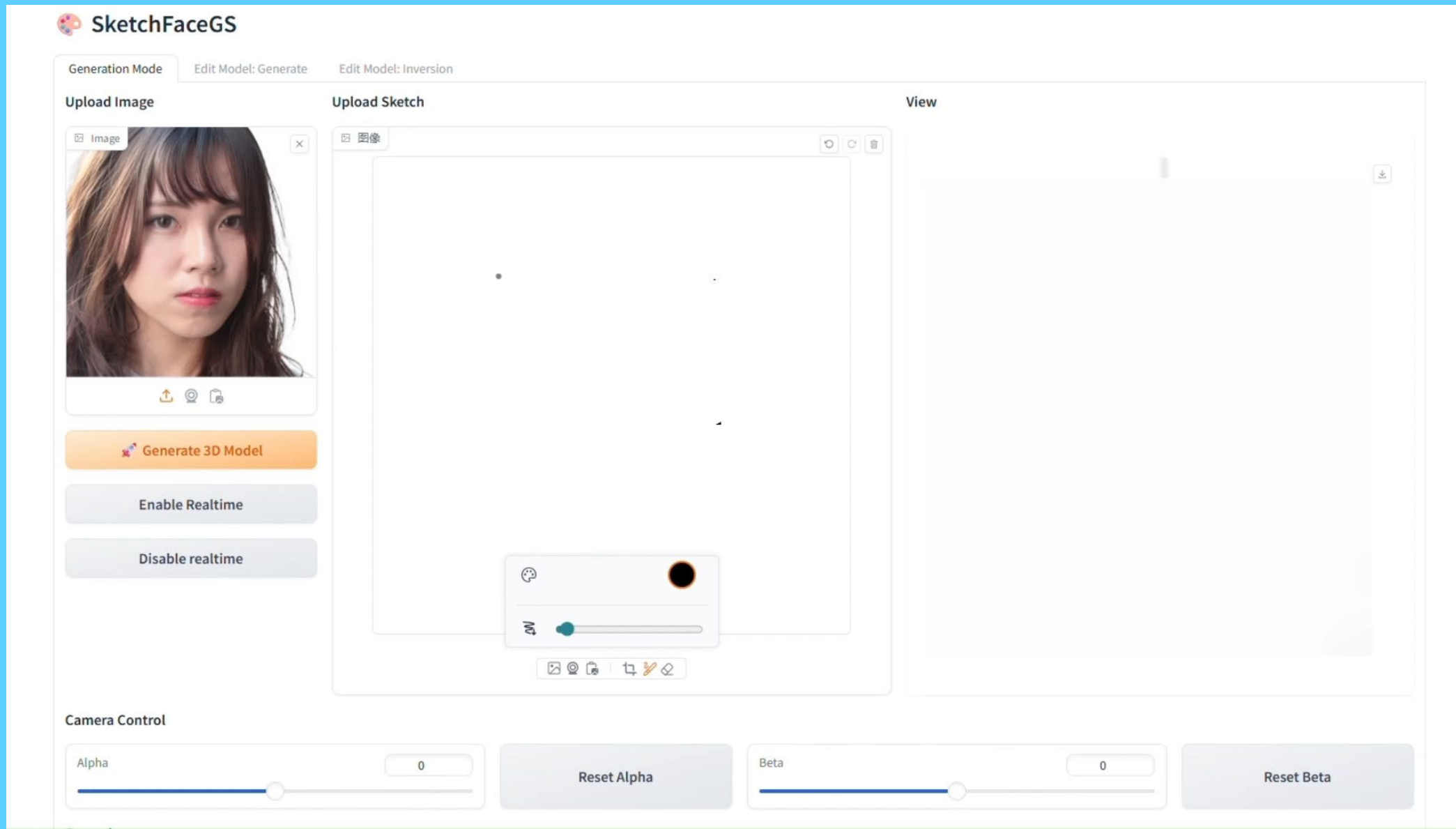


Generate
0.1s

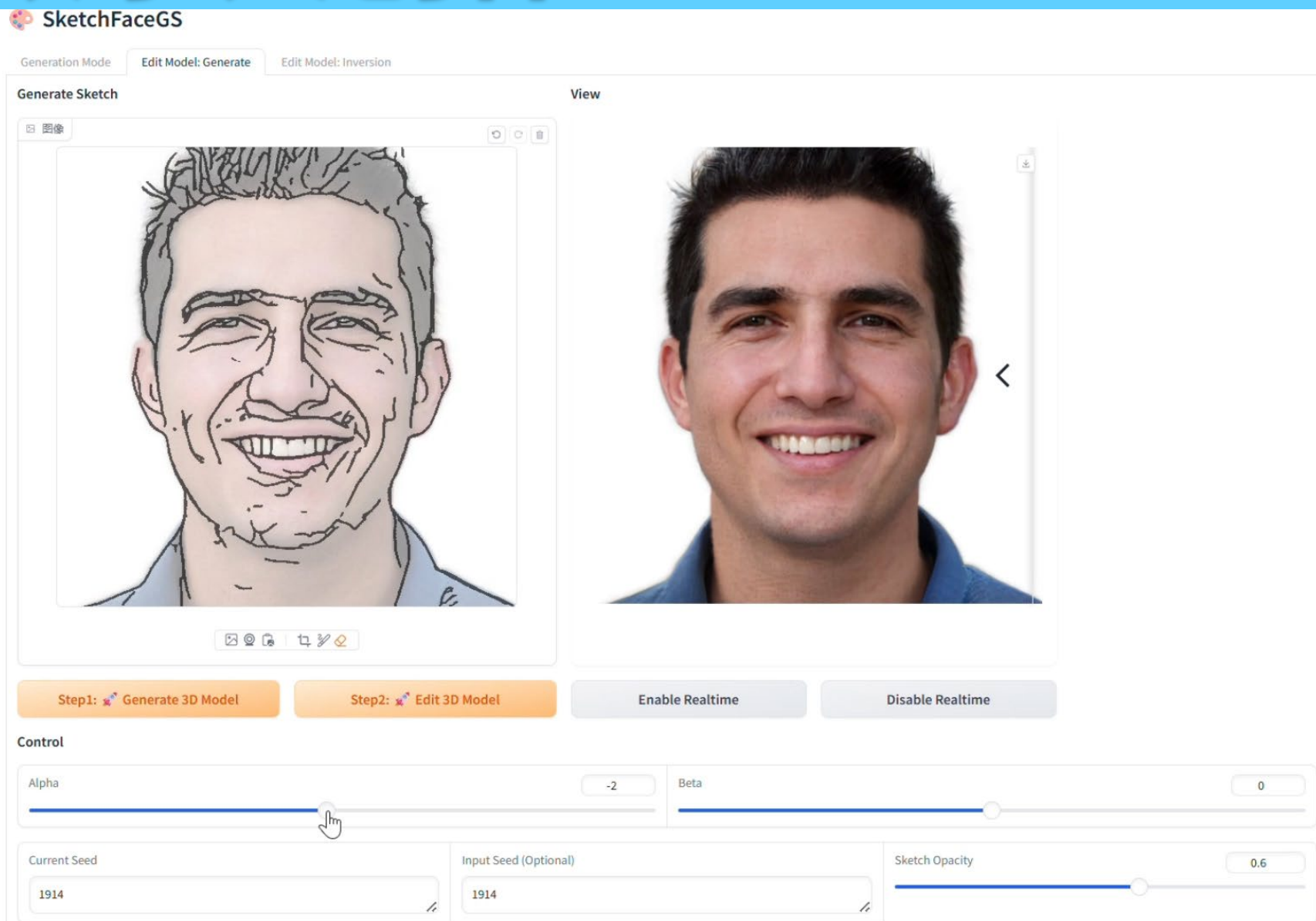


Relight
&
Render
240FPS

三维数字人创作

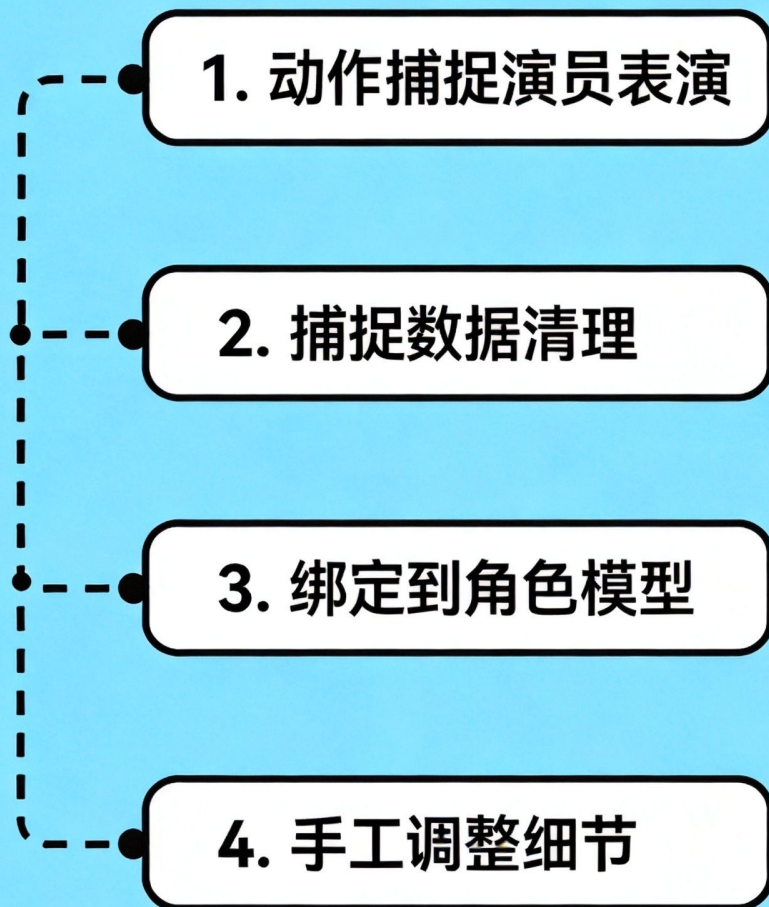


三维数字人创作

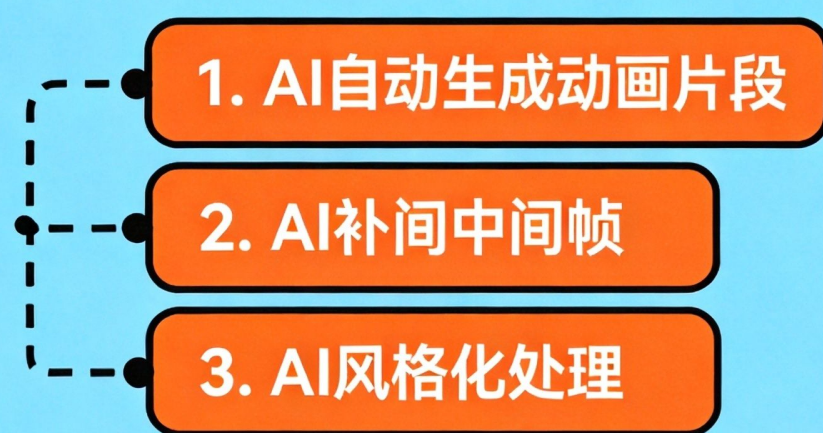


AI辅助动画制作

传统动画制作流程



AI辅助流程



加快制作速度



减少人工成本



提供创意灵感